

DUET

A full-duplex voice agent that acts on what you mean,
not on what you said first.

TECHNICAL DOCUMENTATION

0.81

Pass@1 on all 100
recordings of Full-
Duplex-Bench v3
(best published: 0.60)

0%

interruptions, and
every one of the 100
conversations
answered

1

command to
reproduce: `bash`
`reproduce.sh`

Team SE7EN · SRM Institute of Science and Technology · SRM_SE7EN
Tanmay Singh · Panshul Arora · Naman Rai · Devansh Goenka

<https://github.com/tannnmayy/DUET-SAMSUNGPRISM-HACKATHON-TEAM-SE7EN>

Contents

1. DUET in two minutes

2. Samsung's requirements, and where each is met

The checklist a reviewer can verify against the repository.

3. DUET in one chapter

From the repository's README: results, the model declaration, reproduction, run logs.

4. Architecture

Every component, the turn loop, the coordinator, the model client and the harness.

5. Results

The reported run on all 100 recordings, in detail.

6. Engineering notes

How the benchmark scores, what we measured, and the decision each measurement led to.

7. Use-case extension: DUET for Galaxy

The same agent, coordinator and model on a Samsung phone.

8. Limitations and next steps

What we still lose, and where DUET goes next.

9. Appendix: use-case research

The research behind DUET for Galaxy.

DUET in two minutes

The problem. People interrupt, hesitate and correct themselves mid-sentence. A voice agent that acts on their behalf fails exactly there: the Full-Duplex-Bench v3 paper finds that systems “commit intermediate parameters before the correction arrives.” Under the benchmark’s strict Pass@1, one stale or duplicated tool call fails the whole request.

The solution. DUET is a custom LiveKit voice agent with two minds and a referee. A fast voice keeps the conversation alive; a thinker, Google’s Gemma 4 26B-A4B, reads everything the user has said and plans the tool calls; and a coordinator decides whether and when any action may run. New words void older plans, nothing runs while the user is still talking, and no action ever runs twice.

Pass@1 (strict)	0.81 on all 100 recordings · GPT-Realtime 0.600 · Gemini Live 3.1 0.540
Tool selection · arguments	0.915 · 0.850 (the best published: 0.876 · 0.680)
Self-corrections	15 of 17 pass (0.88; GPT-Realtime 0.588)
Conversations answered · interruptions	100% · 0%
How it was scored	The benchmark’s own runner and scripts; the judge was Gemma 4 standing in for gpt-4o (75 of 100 by exact match with no judge)
Language model	Gemma 4 26B-A4B-it (open weights, Apache 2.0) through Google’s API; seed 7
Use-case extension	DUET for Galaxy: Assistant, Care and Drive modes on a Samsung phone

Reproduce it

```
export GOOGLE_API_KEYS=key1,key2      # keys from Google AI Studio projects without billing
bash reproduce.sh --only travel_19_695bd157114f0d2317f88617  # a quick check
bash reproduce.sh --force              # all 100 recordings, then the official evaluations
```

The submission

Item	Where
Code and README	github.com/tannnmayy/DUET-SAMSUNGPRISM-HACKATHON-TEAM-SE7EN
Presentation	SRM_SE7EN.pptx, SRM_SE7EN.pdf (8 slides)
This documentation	SRM_SE7EN_Documentation.pdf
The reported run	results/reported/gemma4_final_run: reports, 100 traces, logs
Extension demo recordings	results/galaxy/video/
Demo video	Google Drive folder
AI usage disclosure	AI disclosure/LangAI3.0_AI_Disclosure_SE7EN.pdf

Samsung's requirements, and where each is met

The checklist a reviewer can verify against the repository.

Every requirement in Samsung's four documents for this round: the Theme 05 participant guide, the hackathon launch deck, the FAQ, and the AI usage disclosure form. "Linked" marks an item delivered through a link rather than a file in this repository; "At submission", one done when the team submits.

Theme 05 guide: what to submit

Requirement	Where it is met	Status
Code repository with a README covering the architecture (a diagram), exact setup and run steps, and the extension clearly marked	README.md : architecture diagrams, "Reproduce the benchmark", "Use-case extension: DUET for Galaxy". Chapter 3 of this document.	Met
One-command reproduction that runs FDB-v3 end to end (install, configure, evaluate) and declares the model provider	reproduce.sh ; README "Models and providers (declaration)": a custom LiveKit agent, Gemma 4 26B-A4B-it through Google's API.	Met
Benchmark results and run logs (scores, seeds, configuration) from our best run	results/reported/gemma4_final_run : summary.json, run_config.json (seed 7, sampling, every threshold, the git commit), the official reports, 100 per-recording results, a trace of every conversation, the agent, runner and evaluation logs. Chapter 5.	Met
Demo video, 3 to 5 minutes: an interruption on the benchmark, then the extension	The demo video (Google Drive), linked from the README and the submission form. Plan: docs/VIDEO_SCRIPT.md ; the extension's recorded runs: results/galaxy/video .	Linked
Slide deck, at most 8 slides: problem, architecture, results, what next	SRM_SE7EN.pptx and SRM_SE7EN.pdf : 8 slides.	Met
API keys documented, not included	README "API keys": which keys and where they go. Keys are read from the environment or the git-ignored <code>.env.local</code> ; the repository's entire history was scanned and holds none.	Met

Theme 05 guide: how the score is used

Requirement	Where it is met	Status
Benchmark 60%: Samsung re-runs our script on one 48 GB GPU or our declared hosted APIs; only the re-run counts	The declared hosted API is Google's (Gemma 4). The GPU holds only the speech models (2.9 GB at peak). A preflight tests every key and tool calling before anything runs, and every model call survives rate limits, server errors and stalls (Chapter 4, section 7).	Met

Requirement	Where it is met	Status
If the script does not reproduce, that portion scores zero: test it on a clean machine	A fresh clone of the repository on a clean Ubuntu system (1 Oct 2026, the submitted code): installation from the lock files, the pinned benchmark and its data, the speech models and the scoring recognizer all completed. A complete end-to-end run on a fresh clone was verified on an earlier revision.	Partly
Use-case extension 20%: relevant, runs end to end, shown working in the video	DUET for Galaxy (Chapter 7): three modes on the same agent and coordinator; 8 of 8 phone scenarios pass end to end (galaxy_check_scenarios.py).	Met
Documentation, architecture and video 20%: a clear README and an honest architecture explanation	README, ARCHITECTURE.md , RESULTS.md , ENGINEERING_NOTES.md , and this document. Every stand-in and simulation is stated where it applies.	Met
Official evaluation: the LLM judge enabled, one pinned judge for every team; ties break on strict Pass@1	The official scripts run unmodified with <code>--use-llm</code> ; gpt-4o is used whenever <code>OPENAI_API_KEY</code> is set. Our reported run: Pass@1 0.81.	Met

Theme 05 guide: dos and don'ts

Requirement	Where it is met	Status
Cite the public checkpoints and hosted APIs used	README "Models and providers" and "References".	Met
Pin seeds and versions	Every package including transitive ones (<code>requirements*.lock</code>); Hugging Face revisions of Whisper and Kokoro; the Gemma model, its sampling and seed 7 on every call; the benchmark commit, data SHA-256 and LiveKit checksum.	Met
Test the reproduction script on a machine that is not ours	See the clean-machine row above.	Partly
Keep the extension honest	<code>app/README</code> "What is real and what is simulated"; the deck and the video say it on screen.	Met
Don't hardcode, memorise or fine-tune on benchmark test items	Nothing is trained or tuned. test_voice_integrity.py fails if any value the benchmark expects appears in the agent.	Met
Don't call your own servers at evaluation time	The only remote services are Google's API (declared) and, if chosen, LiveKit Cloud.	Met
Don't cache anything across scenarios	A fresh coordinator, toolbox and conversation for every room. The warm-up streams only silence; the benchmark's mock backend is stateless.	Met

Theme 05 brief (launch deck): what to build

Requirement	Where it is met	Status
Keep the conversation alive without losing logical depth	Two minds: a fast voice covers every wait with a fixed line; the thinker (Gemma 4) reasons over the whole request.	Met

Requirement	Where it is met	Status
Handle and recover from real-time interruptions while driving to the goal	Epochs and the commit gate: a plan built on words the user has changed never runs; the next turn re-reads everything.	Met
Changes of goal without losing the session's context	The thinker reads everything said since its last spoken answer, and is told what has already been done in the conversation.	Met
A solid harness for safe, reliable agentic behaviour	The coordinator: commit gate, idempotency ledger, failure policy (reads retried, writes never blindly re-sent).	Met
Full-duplex: begin before the utterance ends; session-scoped memory only	Planning starts at each pause, while the user may still be mid-request, and actions wait at the gate. State lives in the room and ends with it.	Met
Streaming and async designs; latency; multimodal inputs and outputs	Speech, model calls and tools run off the audio loop; latency is measured per recording. Voice in and out; the phone adds an on-screen timeline and device readings. Camera input is not used in Round 1.	Met

Hackathon submission rules (launch deck and FAQ)

Requirement	Where it is met	Status
Working prototype code in a public or shared GitHub repository	github.com/tannnmayy/DUET-SAMSUNGPRISM-HACKATHON-TEAM-SE7EN (public).	Met
README with reproducible setup instructions, Docker files and other requirements	README; Dockerfile (runs the same reproduce.sh); requirements*.lock.	Met
Release tag PRISM_GENAI_HACKATHON_Y2026 on the final commit; everything referenced is in that commit	Created on the final commit when the team submits.	At submission
Demo video, at most 5 minutes (YouTube or Drive link)	Google Drive folder , linked from the README.	Linked
Presentation (PPT or PDF) named CollegeName_TeamName	SRM_SE7EN.pptx and SRM_SE7EN.pdf.	Met
Theme ID, project title, team, problem, solution and architecture, tech stack, innovation, results and limitations in the presentation	Slides 1 to 8 of SRM_SE7EN.	Met
AI usage disclosure form	AI disclosure/LangAI3.0_AI_Disclosure_SE7EN.pdf , also submitted with the form.	Met

Theme 05 FAQ: the organizers' clarifications

Requirement	Where it is met	Status
One GPU with 48 GB (A6000 class); frugal compute is encouraged	2.9 GB of GPU memory at peak in the reported run; the language model runs at Google.	Met
Dependencies from public, safe sources; other domains allowlisted on request	README "Network access" lists every host the reproduction contacts, and why.	Met
A per-scenario wall-clock cap of 300 s	At most 62 s per conversation (median 38 s); 100 recordings in 116 minutes.	Met

DUET in one chapter

From the repository's README: results, the model declaration, reproduction, run logs.

Samsung PRISM GenAI Hackathon 3.0 · Theme 05: Interruptible Real-Time Agents · Team SE7EN, SRM

People interrupt, hesitate and correct themselves mid-sentence: *"Book a flight to Rome, no wait, Milan."* A voice agent that acts on their behalf fails exactly there. The Full-Duplex-Bench v3 paper names the failure that costs every published system the most: *"models commit intermediate parameters before the correction arrives."*

DUET is a LiveKit voice agent built so that this cannot happen, without going quiet while it waits. It pairs two minds, a fast voice that keeps the conversation alive and a thinker (**Gemma 4 26B-A4B**) that reasons over the whole request and calls the tools, with a **coordinator** that decides *when acting is allowed*.

Benchmark	Full-Duplex-Bench v3: 100 real recordings, 12 tools, four domains, the official runner and scripts
Language model	Gemma 4 26B-A4B-it (open weights, Apache 2.0) through Google's API
Speech	faster-whisper large-v3-turbo, Silero VAD, LiveKit end-of-turn model, Kokoro-82M, all local
Use-case extension	DUET for Galaxy: the same agent on a Samsung phone, in three modes (app/)
Demo video	Google Drive folder
AI disclosure	AI disclosure/LangAI3.0_AI_Disclosure_SE7EN.pdf
Reproduction	One command, <code>bash reproduce.sh</code> (below)

Results

Full-Duplex-Bench v3, all 100 recordings, streamed in real time through the benchmark's own runner and scored by its own scripts. The complete record of the run, with every trace and log, is in [results/reported/gemma4_final_run](#).

System	Pass@1	Tool selection	Argument accuracy	Response quality	Turn-take	Task latency	Interruptions
DUET, Gemma 4 26B-A4B (ours)	0.81	0.915	0.850	0.730	100%	14.44 s	0%
GPT-Realtime (paper)	0.600	0.876	0.680	0.792	96.0%	6.89 s	13.5%
Gemini Live 3.1 (paper)	0.540	0.817	0.588	0.718	78.0%	4.25 s	19.2%
Cascaded Whisper → GPT-4o → TTS (paper)	0.450	0.803	0.562	0.600	100%	10.12 s	33.0%

Scored by the benchmark's own scripts on our laptop (RTX 3060, Windows) with two disclosed stand-ins: the LLM judge is Gemma 4 26B-A4B answering the official gpt-4o prompts unchanged, and faster-whisper transcribes DUET's own speech in place of the Linux-only Parakeet (this affects response quality and latency, not the tool

metrics). With no judge at all, by exact match, DUET passes 75 of 100. Samsung's re-run uses the official gpt-4o judge and Parakeet.

How we measured, what the remaining misses are, and how DUET behaves on each kind of disfluency: [docs/RESULTS.md](#).

Models and providers (declaration)

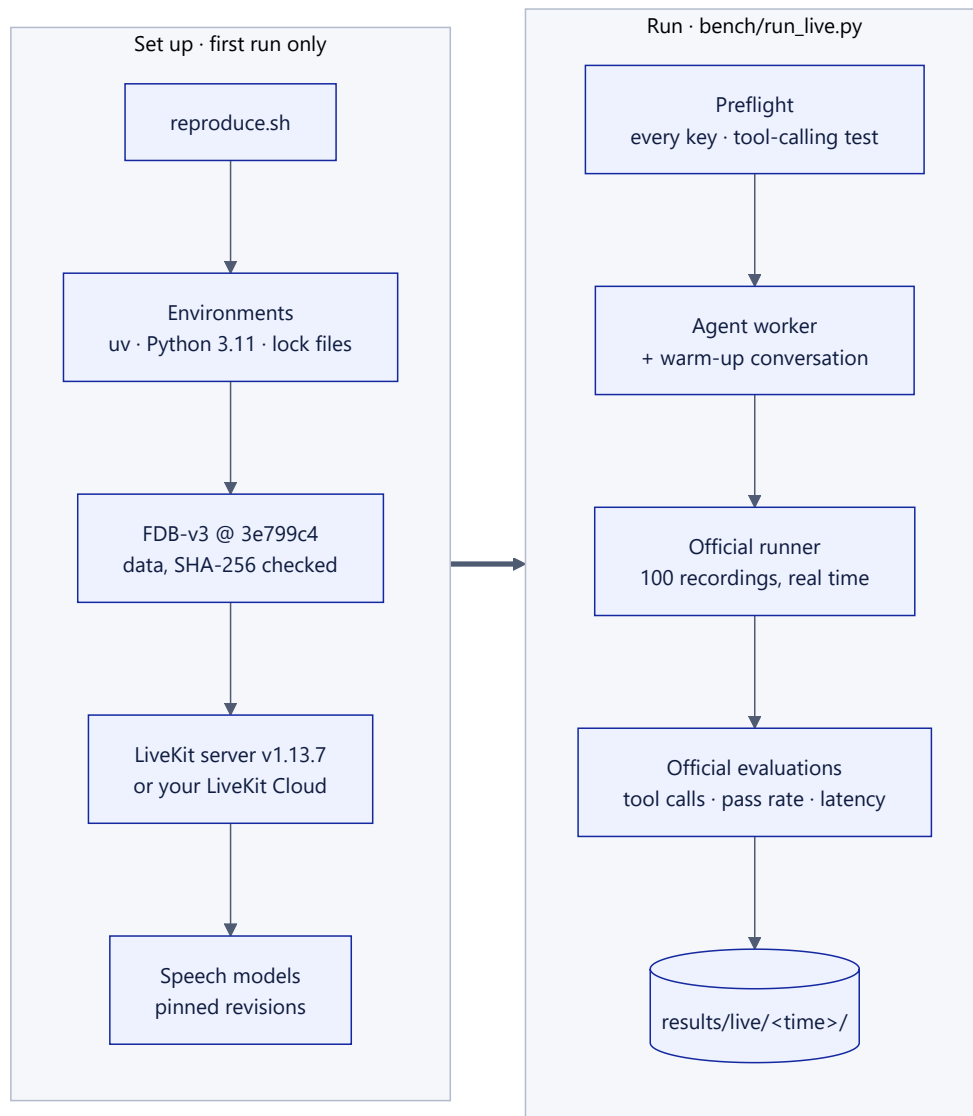
DUET is a **custom LiveKit agent**, not one of the benchmark's realtime-provider presets.

Role	Model	Where it runs
Thinker: reasoning and tool calls	Gemma 4 26B-A4B-it (<code>gemma-4-26b-a4b-it</code>), open weights, Apache 2.0. <code>gemma-4-31b-it</code> is the fallback if a key is not served the 26B. Sampling as Google recommends for Gemma 4 (temperature 1.0, top-p 0.95, top-k 64), seed 7 on every call	Google's API (<code>google-gemai-2.25.0</code>)
Fast voice	Fixed lines, no model	Local
Speech recognition	faster-whisper large-v3-turbo (<code>mobiuslabsgmbh/faster-whisper-large-v3-turbo @ 0a363e9</code> , float16)	Local GPU
Voice activity	Silero VAD (LiveKit plugin)	Local CPU
End of turn	LiveKit <code>turn-detector-v1-mini</code> (audio model)	Local CPU
Text-to-speech	Kokoro-82M (<code>hexgrad/Kokoro-82M @ f3ff357</code> , voice <code>af_heart</code>)	Local GPU
Tool backend	The benchmark's own <code>mock_apis.py</code> , unmodified	Local
LLM judge	The benchmark's gpt-4o when an <code>OPENAI_API_KEY</code> is set; otherwise Gemma 4 26B-A4B answers the same prompts and every report says so	OpenAI / Google

Why Gemma 4 26B-A4B. A mixture-of-experts model (25.2B parameters, 3.8B active per token) with native function calling, strong on tool use (68.2 on τ^2 -bench), and open weights anyone can inspect. Served through Google's API, the re-run downloads no large model: the GPU runs only the speech models, a few GB, far inside Samsung's 48 GB card.

Reproduce the benchmark

```
export GOOGLE_API_KEYS=key1,key2          # or GOOGLE_API_KEY=key (also read from .env.local)
bash scripts/doctor.sh                     # optional: is this machine ready? (checks the keys too)
bash reproduce.sh --only travel_19_695bd157114f0d2317f88617 # a two-minute check
bash reproduce.sh --force                   # all 100 recordings, then the three official evaluations
```



- **Machine:** Linux x86_64, an NVIDIA GPU with about 8 GB free and a CUDA 12.x or 13.x driver, 16 GB of RAM or more (loading the benchmark's NeMo scoring recognizer takes several GB on top of the agent's own), `git`, `curl`, internet access, about 30 GB of disk. No sudo, no system Python: a pinned `uv` supplies Python 3.11; `ffmpeg` and the LiveKit server are fetched into `third_party/`.
- **Time:** about two hours for all 100 recordings (the benchmark streams each one in real time), plus about 15 minutes of installation on the first run.
- **What it runs:** the benchmark's **unmodified** `run_tool_benchmark_all_released.py --provider duet`, then its three evaluation scripts with `--use-11m`, and prints the headline numbers and the GPU's peak memory.
- `--force` re-runs recordings that already have a result; use it on every full run after the quick check.

API keys

No key is included in this repository. Set them in the environment or in `.env.local` at the repository root (git-ignored).

Variable	Purpose	Required
<code>GOOGLE_API_KEY</code>	Gemma 4 through Google's API. Create one at aistudio.google.com (Get API key) in a project without billing : Google serves Gemma free of charge on the free tier. The preflight tests every key before the run starts and names any problem	Yes (or the next line)
<code>GOOGLE_API_KEYS</code>	Several keys, comma-separated, each from a different project without billing. Their per-minute limits add up; two or three are recommended for a full run	Recommended
<code>OPENAI_API_KEY</code>	The benchmark's gpt-4o judge (argument and response scoring, key-information latency)	Optional
<code>LIVEKIT_URL</code> , <code>LIVEKIT_API_KEY</code> , <code>LIVEKIT_API_SECRET</code>	A LiveKit Cloud project instead of the local LiveKit server	Optional

In a container

The `Dockerfile` runs the same `reproduce.sh` on a clean Ubuntu 22.04 image, adding only the tools the script expects (git, curl, xz). It needs the NVIDIA Container Toolkit on the host.

```
docker build -t duet .
docker run --rm --gpus all --shm-size 8g -e GOOGLE_API_KEYS=key1,key2 \
  -v duet-cache:/duet/third_party -v "$PWD/results/live:/duet/results/live" \
  duet --only travel_19_695bd157114f0d2317f88617      # or --force for all 100 recordings
```

Network access

For a machine behind a firewall, these are every host the reproduction contacts:

Host	What for	When
<code>generativelanguage.googleapis.com</code>	Gemma 4, the language model	Every run
<code>api.openai.com</code>	The official gpt-4o judge, when <code>OPENAI_API_KEY</code> is set	Evaluation
<code>pypi.org</code> , <code>files.pythonhosted.org</code>	Python packages, from the lock files	First run
<code>huggingface.co</code> and its download hosts (<code>*.huggingface.co</code> , <code>*.hf.co</code>)	The speech models and the benchmark's Parakeet recognizer	First run
<code>github.com</code> and its download hosts (<code>objects.githubusercontent.com</code> , <code>release-assets.githubusercontent.com</code>)	Full-Duplex-Bench, the LiveKit server, ffmpeg, Python 3.11 builds, one spaCy model	First run
<code>astral.sh</code> , <code>releases.astral.sh</code>	The pinned <code>uv</code> , if it is not installed	First run
<code>drive.usercontent.google.com</code>	The benchmark audio, unless <code>FDB_DATA_DIR</code> points at a copy	First run

`bash scripts/doctor.sh` checks that the main one in each row can be reached (all but the optional judge's).

Run it under your own harness

The agent is an ordinary LiveKit worker. It joins every new room in the project (no agent name, no explicit dispatch), so run only this worker there.

```
.venv/bin/python -m duet_voice.prefetch      # once: download and warm the speech models
.venv/bin/python -m duet_voice.gemma_api     # the keys reach Gemma 4 and tool calling works
.venv/bin/python -m duet_voice.agent start   # ready when the log shows "registered worker"
```

Then run the benchmark's own runner and evaluations with `--provider duet`. Tool calls go to `/tmp/agent_tool_calls.log` in the benchmark's format.

Run logs

Every run writes `results/live/<time>/`; the run reported above is copied, without audio, to `results/reported/` by `bench/publish_run.py`.

File	Content
<code>run_config.json</code>	The agent's effective settings (model, sampling, seed, keys, token budget, listening thresholds), overrides, the LiveKit target, the scoring recognizer, the code version and the machine
<code>summary.json</code>	Headline metrics, what the agent did (keep-listening decisions, tool calls, model errors), tokens, and the GPU's peak memory
<code>duet_*_report.json</code>	The benchmark's own evaluation, pass-rate and latency reports
<code>items/*.json</code>	The benchmark's per-recording results: transcripts, tool calls, timings
<code>traces/*.jsonl</code>	DUET's own trace of each conversation: every heard segment, turn decision, thinker step, tool call with arguments and outcome, token counts, and speech timings
<code>agent.log</code> , <code>runner.log</code> , <code>eval_*.log</code>	Process logs, including one line per model request

Engineering quality

- **75 unit tests** (`python -m pytest tests -q`): the coordinator's gate, epochs and ledger; the key pool, backups and deadlines; the speaking rules; tool argument handling; the phone bridge.
- **No benchmark item is written into the agent.** `tests/test_voice_integrity.py` fails if any argument value the benchmark expects appears in any string the agent contains. Prompts and tool schemas are written from general principles, and nothing is trained or tuned on the benchmark.
- **Nothing is cached across scenarios.** Each LiveKit room gets a fresh coordinator, toolbox and conversation.
- **Pinned:** every Python package including transitive ones (`requirements*.lock`), the benchmark commit, the data checksum, the LiveKit server version, the Hugging Face revisions of the speech models, the model name, its sampling and a fixed seed.

- **Robust on someone else's machine:** the preflight names any problem with a key before anything runs; the local LiveKit server takes free ports and never reuses another user's; downloads and caches stay inside the repository folder.
- **Tool schemas** keep the benchmark's names, arguments and log format. Arguments a real API treats as optional stay optional, and values are cleaned the way an API takes them (numbers as numbers, dates as month and day, spelled-out ids joined).

Repository map

duet_voice/	the agent
agent.py	LiveKit endpoint: ears, fast voice, the turn loop
coordinator.py	epochs, commit gate, idempotency ledger, failure policy
thinker.py	Gemma 4 tool loop, keep-listening
gemma_api.py	key pool, token budget, retries, backup requests, preflight
fdb_tools.py	the 12 FDB-v3 tools over the benchmark's mock backend
prompts.py	the thinker's instructions
galaxy/	the use-case extension's agent: phone tools, three modes
bench/	live runner, warm-up, offline evaluators, judge adapter, reports
scripts/	machine check, offline evaluation, the extension's demo tools
app/	DUET for Galaxy: Android app, web build, demo recorder
tests/	75 unit tests
docs/	architecture, results, engineering notes, requirements, research
AI disclosure/	LangAI 3.0 AI disclosure form (PDF)
results/reported/	the reported run: reports, traces and logs
reproduce.sh	one-command reproduction
legacy/	DUET v1 on the original Theme 05 kit (research record, not used)

References

- G.-T. Lin, C. Chen, Z. Chen, H.-y. Lee. *Full-Duplex-Bench-v3: Benchmarking Tool Use for Full-Duplex Voice Agents Under Real-World Disfluency*. arXiv:2604.04847, 2026. Code and data: <https://github.com/DanielLin94144/Full-Duplex-Bench> (v3).
- Gemma 4: <https://ai.google.dev/gemma> · LiveKit Agents: <https://github.com/livekit/agents> · faster-whisper: <https://github.com/SYSTRAN/faster-whisper> · Kokoro-82M: <https://huggingface.co/hexgrad/Kokoro-82M> · Silero VAD: <https://github.com/snakers4/silero-vad>

Architecture

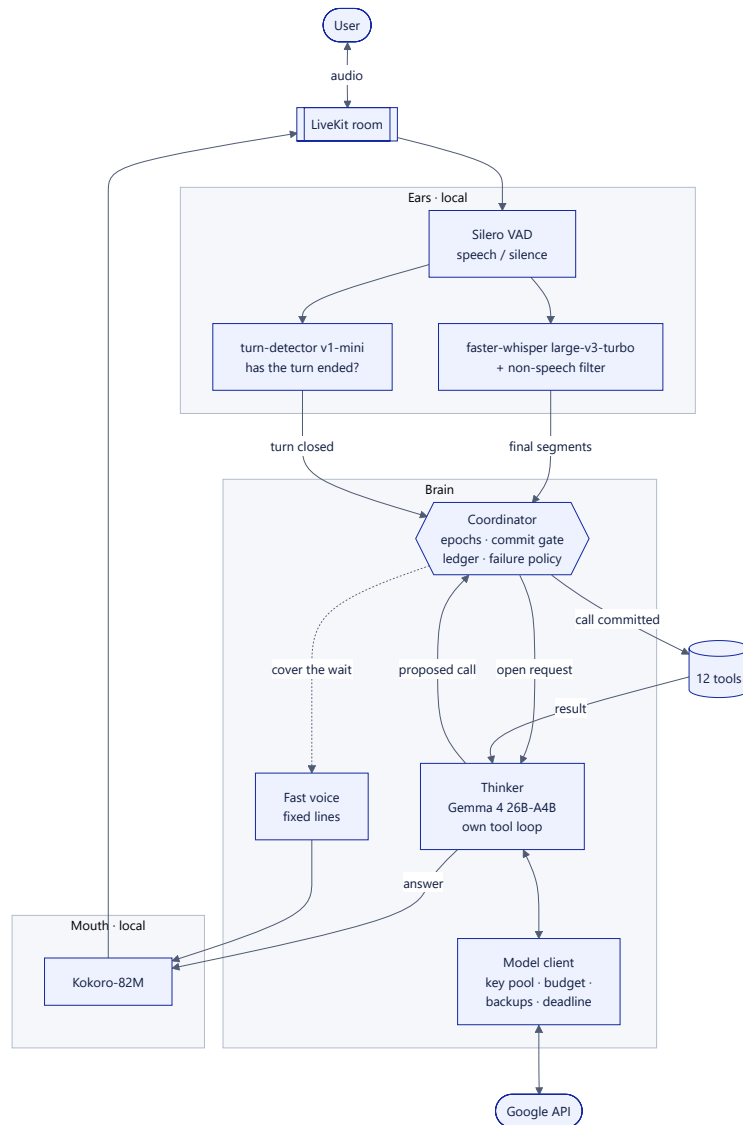
Every component, the turn loop, the coordinator, the model client and the harness.

DUET is a voice agent built for one hard problem: **acting on what a person means while they are still changing their mind**. The Full-Duplex-Bench v3 paper names the failure that costs every published system the most: *"models commit intermediate parameters before the correction arrives."* DUET is organised around never doing that, without leaving the user in silence.

This document describes every part of the system, why it exists, and the thresholds it uses. The README gives the short version.

- 1. The system at a glance
- 2. Where everything runs
- 3. The ears
- 4. The turn loop
- 5. The coordinator
- 6. The thinker
- 7. The model client
- 8. The tools
- 9. The benchmark harness
- 10. DUET for Galaxy
- 11. Latency and resources
- 12. Configuration

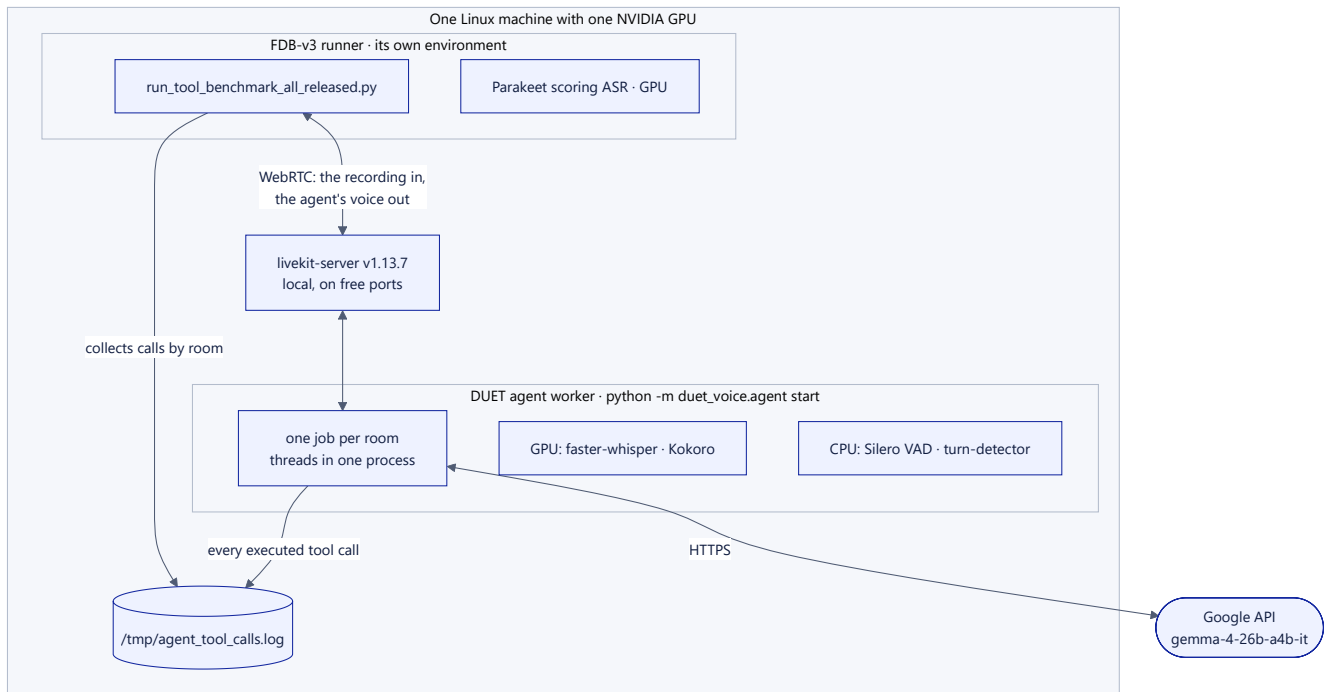
1. The system at a glance



Three ideas carry the design:

1. **Two minds.** A thinker that is allowed to be slow and careful, and a fast voice that never lets the conversation go silent. The fast voice says only things that stay true whatever the user says next.
2. **One owner for every action.** The model proposes tool calls; the coordinator decides whether and when each one runs. The model can be wrong about timing without the user ever paying for it.
3. **The whole request, every time.** The thinker always reads everything the user has said since the last answer that was actually spoken, so a correction is never read without the words it corrects.

2. Where everything runs



- Conversations run as **threads of one worker process**, so the speech models are loaded once and shared, however many rooms come and go. Model calls release the GIL and never block the audio loop.
- The worker reports its **load in conversations, not CPU**: between recordings the benchmark's own scoring recognizer loads the CPU, and LiveKit's default CPU-based load would make the worker refuse rooms.
- The only remote service is **Google's API** for the language model (and LiveKit Cloud, if a project is configured instead of the local server).

3. The ears

Stage	Component	Setting	Why
Voice activity	Silero VAD	0.35 s minimum silence per segment	Short enough to see a pause, long enough to ignore a breath
Recognition	faster-whisper large-v3-turbo, float16 on the GPU, one VAD segment at a time, greedy/beam decoding	A prompt listing the tools' vocabulary	Deterministic re-runs; ids, currencies and document names are heard more often
Non-speech filter	Whisper's own confidences	Drop a segment with no-speech probability > 0.6 and mean log-probability < -1.0, or a degenerate compression ratio	Every FDB-v3 request is followed by ~30 s of the speaker's real room noise, exactly where Whisper invents sentences, and an invented sentence can trigger an extra, failing call
End of turn	LiveKit <code>turn-detector-v1-mini</code> , an audio model, locally	0.8 s minimum, 2.5 s maximum endpointing delay	Closes a turn quickly when the user has clearly finished

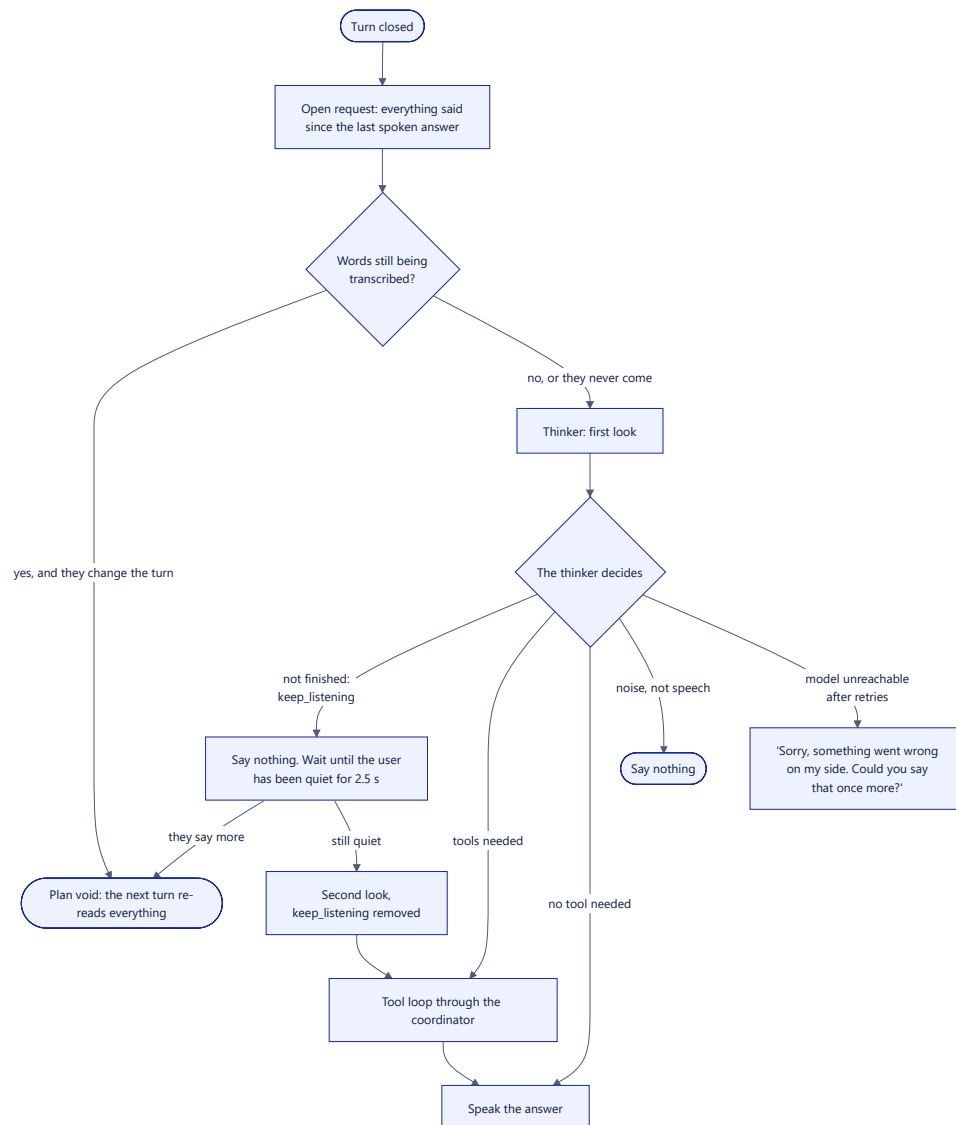
The ears only report what they heard and when a turn seems to have ended. Whether the user has *really* finished is decided later, by the thinker (section 6), and whether an action may run is decided by the coordinator (section 5).

Late transcripts. The end-of-turn model judges the audio, and it can close a turn while Whisper is still transcribing the last segment (in our laptop runs, about a third of all turns). The words that arrive afterwards are the end of the user's sentence, often a correction: "...pay it from checking. Wait, no, make it savings." Two rules make this harmless:

- words that arrive after a turn closed advance the epoch, exactly like new speech, so any plan made without them is void (section 5);
- before the thinker spends a model call on a closed turn, it waits for a transcript that is still due, at most 1.5 s after the speech ended (`Coordinator.settle`). A turn whose words are all in waits for nothing.

4. The turn loop

Every closed turn runs DUET's `llm_node` (`duet_voice/agent.py`), which replaces LiveKit's default LLM step with the two minds.



What the user hears, and when

Moment	What is said	Rule
The user has clearly finished (quiet for 1.6 s) and the thinker is still working	"One moment."	The benchmark ends a turn at the first silence longer than 2 s, so this can never land inside the user's turn
The thinker decides a tool is needed, if nothing was said yet	"One moment."	Never twice
Tools still running	"Still working on it." every 6 s	No dead air on long chains
After the tools	The thinker's answer: one or two sentences with the key facts from the results	Nothing is announced as done before its tool result
The thinker is waiting for the user to finish (<code>keep_listening</code>)	Nothing	A half-heard request is neither acted on nor answered
The model is unreachable after every retry	A short apology	The user is never left in silence

The fast voice never names a value: the user may still be correcting it, and a stale value spoken aloud is a claim. It never states a result either.

When a request counts as answered. The open request is closed only when a reply that finished thinking has actually been spoken. A reply drafted during a pause and then thrown away leaves the request open, so the real turn is always answered (`DuetAgent.reply_delivered`).

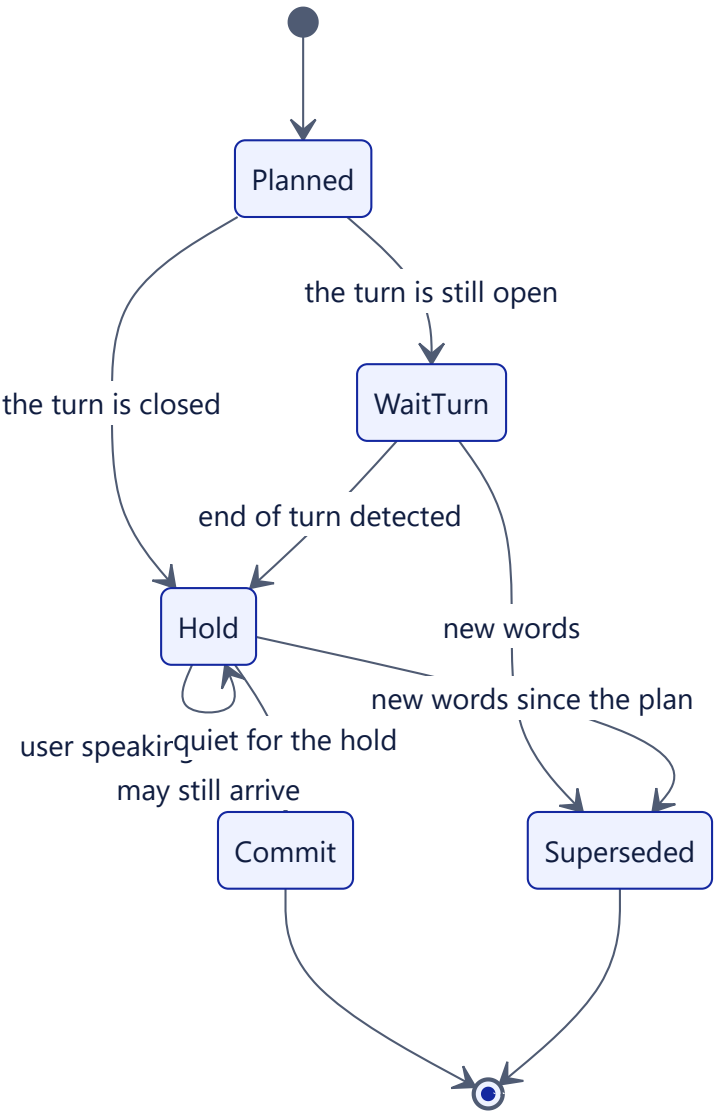
5. The coordinator

`duet_voice/coordinator.py` owns every action. It is small, synchronous in spirit, and covered by its own tests (`tests/test_voice_coordinator.py`).

Epochs

The epoch is a counter of *what the user has said*. It advances when the user starts speaking, and when words arrive after a turn was closed. Every plan is stamped with the epoch it was made in, and every tool call carries that stamp to the gate.

The commit gate



A call passes the gate only when **the turn it was planned in has closed, no new words have arrived since, and the user has been quiet for a hold** that depends on how their last words ended:

The user's last words	Hold	Example
A plain request	1.1 s	"Track order X K 4 2."
A revision somewhere in the request	1.8 s	"...no wait", "actually", "I mean", "make it", "instead"
A sentence left open	2.2 s	"...and", "um", "the", "let me think", a trailing comma

The holds only apply to acting. Thinking starts as soon as the turn closes, so the hold mostly overlaps the model call.

The idempotency ledger

Every call that reaches the backend is recorded under a canonical key: the tool plus its arguments, with case, order, spacing and nulls normalised. An identical call later in the conversation is answered from the ledger (`already_done`) instead of being sent again. For a state-changing tool this is the guarantee **never perform the**

same action twice.

A call that has passed the gate is **shielded**: if the user barges in and the reply that owned the call is cancelled, the call still runs to completion and is recorded. Dropping it would leave an action that happened unaccounted for, and a re-plan could perform it again.

The failure policy

Failure	Read-only tool	State-changing tool
The backend raised an error	Retried once	Not retried; the thinker says what failed and offers a human agent
The backend timed out	Retried once	Recorded as unknown and never re-sent: it may have happened

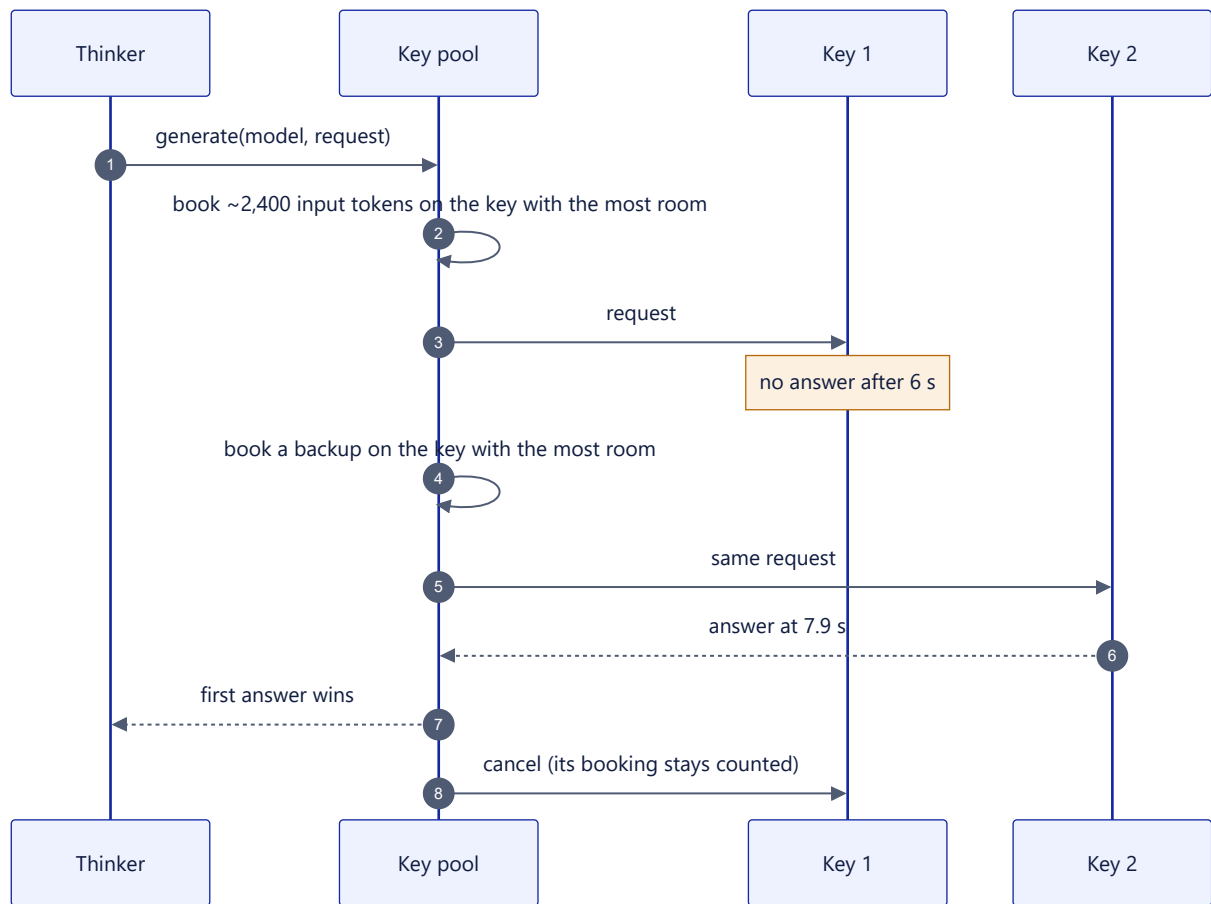
6. The thinker

`duet_voice/thinker.py` runs Gemma 4 26B-A4B with DUET's own tool loop rather than the framework's, so that every call goes through the coordinator.

- **Instructions** (`duet_voice/prompts.py`) teach reading, not the benchmark: ignore fillers and restarts; a self-correction replaces what came before, including several corrections in one request; a dropped request is not performed; a spelled-out id is one token; carry out every task asked, in order, one call per action; decide conditions on the actual tool results; pass optional arguments only when the user gave them; never claim what a tool result does not say.
- **Native conversation state**. The model's turns are kept exactly as returned, including thoughts and their signatures, so follow-up requests in a tool chain are accepted, and a `keep_listening` next to real work is answered rather than deleted.
- **keep_listening**. On the first look at a turn the thinker has one extra tool: *"call this instead of acting or answering when the user has clearly not finished speaking"*. Nothing is done or remembered; once the user has been quiet for 2.5 s the thinker looks again without it and must act or ask for exactly what is missing.
- **Empty or malformed replies** are asked for again once before giving up.
- **Tool chains** of up to 8 steps: when a step needs something an earlier step returns (an id, an address, a price), the earlier tool is called first and its result used.
- **Sampling** as Google recommends for Gemma 4 (temperature 1.0, top-p 0.95, top-k 64), with a fixed seed on every call. Gemma 4 always reasons before it answers.

7. The model client

`duet_voice/gemma_api.py` makes every model call dependable inside a recording's answer window of about 30 s.



Mechanism	Setting	What it prevents
Key pool	<code>GOOGLE_API_KEYS=key1, key2, ...</code>	Keys from different projects add up their per-minute limits; each call goes to the key with the most room
Token budget	15,000 input tokens per key, model and minute (<code>DUET_GEMMA_TPM</code> ; Google allows 16,000)	A call is booked before it is sent and waits for room instead of being refused. One thinker call is about 2,300-2,600 input tokens
Rate limits	On a 429, the key rests for the delay Google asks for	The call moves to another key at once
Transient errors	500, 503, 504: retried after 0.3-0.7 s	Brief server errors never reach the user
Backup requests	A request unanswered after 6 s (<code>DUET_GEMMA_HEDGE_S</code>) gets a backup on the key with the most room; up to 4 in flight; the first answer wins	A request that stalls. Answers take about 3 s at the median
Deadline	28 s per call (<code>DUET_GEMMA_DEADLINE_S</code>)	Past it, the recording's room has closed anyway
Fallback model	<code>gemma-4-31b-it</code> if a key is not served the 26B	A model withdrawn from a key

Mechanism	Setting	What it prevents
Preflight	<code>python -m duet_voice.gemma_api</code>	Every key is probed and a real tool call with a correction is checked ("K 7, no wait, K 4 Q 2" must give K4Q2) before a run starts

Every request is logged in `agent.log` : which key, how long, how many input tokens, and every refusal, retry, backup and cancellation.

8. The tools

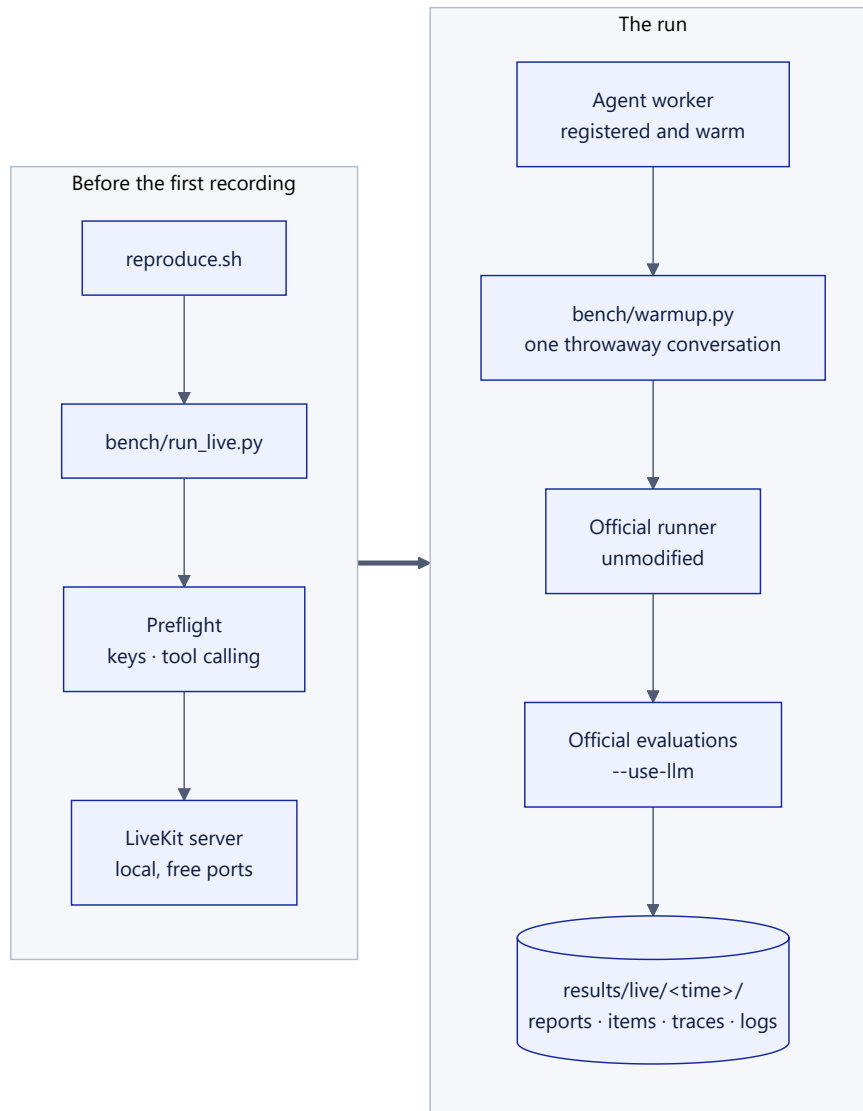
`duet_voice/fdb_tools.py` exposes the benchmark's twelve tools with the benchmark's names, argument names and call-log format, over the benchmark's own `mock_apis.py`.

Domain	Tools
Travel and identity	<code>search_flights</code> , <code>book_flight</code> , <code>update_identity_doc</code>
Finance and billing	<code>get_card_benefits</code> , <code>get_exchange_rate</code> , <code>modify_autopay</code>
Housing and location	<code>search_apartments</code> , <code>calculate_commute</code> , <code>update_search_filter</code>
E-commerce	<code>track_order</code> , <code>search_products</code> , <code>add_to_cart</code>

Four choices a real voice agent would make:

1. **Optional arguments stay optional.** A real apartment search does not need a budget; the schema does not force the model to invent one.
2. **Arguments are cleaned the way an API takes them.** A filter value is a number, a yes/no or text; a date is month and day ("October 4"); an id the user spelled out is joined ("Q-4" is "Q4"); a price the user's plan depends on ("if one is under \$X, add it") is passed as the search's cap.
3. **Calls go through the coordinator,** and the backend runs in a worker thread, so a slow backend never freezes the audio loop.
4. **The log records what the agent asked for.** Where the mock's Python signature needs a value the schema leaves optional, the adapter passes a neutral default; the logged call keeps exactly the agent's arguments.

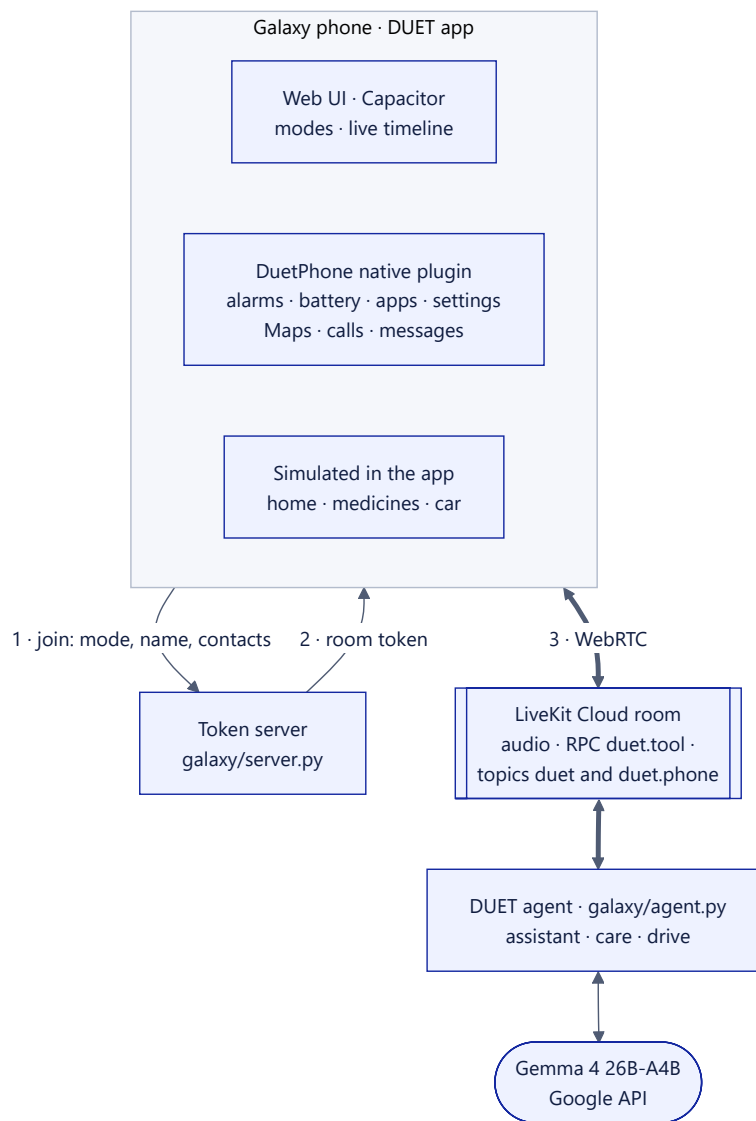
9. The benchmark harness



- **Warm-up.** The first room a worker process joins pays one-time costs (starting WebRTC, the first session's objects). The benchmark's runner starts streaming 2 s after it has joined, without waiting for the agent, so the warm-up conversation makes sure the first recording never pays them.
- **The official scripts run unmodified.** `bench/official_eval.py` runs each evaluation script as it is. With a working `OPENAI_API_KEY` that is exactly the official command (gpt-4o judge). Without one, the scripts' OpenAI client is pointed at Gemma 4 through Google's API answering the same prompts, and every report is labelled accordingly.
- **Continuation.** A run that stopped part-way continues where it left off; a recording that was recorded but never scored is run again.
- **The record.** `run_config.json` holds the agent's effective settings, overrides, the code version and the machine; `summary.json` the headline metrics, what the agent did and the GPU's peak memory.

10. DUET for Galaxy

The use-case extension (`duet_voice/galaxy/`, `app/`) runs the same agent on a Samsung Galaxy phone. Nothing in it is used by the benchmark run.



- **Three modes**, each with its own tools and instructions (`duet_voice/galaxy/modes.py`):

Mode	Tools
Assistant	alarms, timers, battery report, app usage, installed apps, brightness, screen timeout, settings, apps, Maps navigation, calls, messages, the home
Care	medication schedule and log, reminders, calls, messages, family alert, daily check-in, the home
Drive	trip status, destination, stops, navigation, car climate, messages, calls, the home

- **The same coordinator** gates every phone action: no action while the user is speaking, a plan their correction overtook never reaches the phone, and an action is never repeated. Actions that can meaningfully happen again (a new timer, a new message) may run again in a new request, never twice within one.
- **Context the benchmark does not need:** the thinker knows the time; when the user cut an answer short, it knows how much of it they heard; a plan that never ran is reported as never carried out, so the thinker never claims it; an action that did run before the user's latest words is reported as done, so the thinker can undo it if they took it back.

- **The live timeline** on the phone shows each step as it happens: heard, planned, **Never ran**, **Done**, **Not repeated**, and what DUET said.

11. Latency and resources

A typical turn with one tool, measured from the user's last word (medians from our runs on a laptop RTX 3060; a data-centre GPU transcribes faster):

Time	What happens
~0.7 s	The VAD notices the silence
~2.1 s	The last segment is transcribed and the end of turn decided: the thinker starts
~2.3 s	"One moment." starts: the user has now been quiet for 1.6 s
~5 s	The thinker's decision (one Gemma 4 call, ~3 s at the median). The gate's hold has already passed while it thought, so the tool runs at once
~8 s	The answer with the key facts starts (a second, shorter call)

The benchmark's recorder adds a near-constant ~1.9 s to every system's measured latency: the agent's audio frames queued during the 2 s before streaming starts are written first.

GPU memory: the agent's speech models (Whisper large-v3-turbo in float16, Kokoro) and the benchmark's own scoring recognizer, a few GB in all. The language model runs at Google, so Samsung's 48 GB card is far more than enough. `summary.json` records the peak of each run.

12. Configuration

Every setting has a default and an environment override (`duet_voice/config.py`); each run records the effective values in `run_config.json`.

Variable	Default	Meaning
<code>GOOGLE_API_KEYS</code> / <code>GOOGLE_API_KEY</code>	(required)	Keys for Google's API
<code>DUET_THINKER_MODEL</code>	<code>gemma-4-26b-a4b-it</code>	The thinker
<code>DUET_SEED</code>	7	Fixed seed on every model call
<code>DUET_GEMMA_TPM</code>	15000	Input tokens per key, model and minute
<code>DUET_GEMMA_DEADLINE_S</code>	28	Deadline per model call
<code>DUET_GEMMA_HEDGE_S</code>	6	Seconds before a backup request
<code>DUET_COMMIT_HOLD</code> / <code>DUET_REVISING_HOLD</code> / <code>DUET_DANGLING_HOLD</code>	1.1 / 1.8 / 2.2	The gate's holds

Variable	Default	Meaning
DUET_LISTEN_WAIT	2.5	Quiet time before the thinker answers after keep_listening
DUET_ACK_MIN_QUIET	1.6	Quiet time before the fast voice covers a slow thinker
DUET_ENDPOINT_MIN / DUET_ENDPOINT_MAX	0.8 / 2.5	End-of-turn delays
DUET_VAD_MIN_SILENCE	0.35	VAD segment silence
DUET_ASR_MODEL	large-v3-turbo	Whisper model
DUET_TTS_VOICE	af_heart	Kokoro voice
DUET_JUDGE	auto	openai , gemma or none for our own evaluations

Results

The reported run on all 100 recordings, in detail.

The reported run: **all 100 recordings of Full-Duplex-Bench v3**, streamed in real time through the benchmark's own runner and scored by its own evaluation scripts, on the code in this repository (`e6c1b48`). The complete record, with every report, per-recording result, conversation trace and log, is [results/reported/gemma4_final_run](#).

Headline

Metric	DUET	GPT-Realtime (paper)
Pass@1 , strict: every expected tool, no extra call, every argument right	0.81 (81 of 100)	0.600
Tool selection	0.915	0.876
Argument accuracy	0.850	0.680
Response quality	0.730	0.792
Conversations answered (turn-take)	100%	96.0%
Interruptions	0%	13.5%
First response, as the benchmark measures it	4.80 s	
First tool call	9.78 s	
Key information spoken (task completion)	14.44 s	6.89 s

Judged by Gemma 4 26B-A4B standing in for gpt-4o. By exact match, with no judge, DUET passes 75 of 100.

How it was measured

- **The benchmark's pipeline, unmodified.** `bash reproduce.sh --force` runs `run_tool_benchmark_all_released.py --provider duet` and then `evaluate_tool_calls.py`, `evaluate_pass_rate.py` and `analyze_tool_latency.py` with `--use-llm`.
- **Tool metrics** come from the agent's call log exactly as the benchmark reads it (`/tmp/agent_tool_calls.log`): every call DUET actually made, with its arguments.
- **The machine:** a laptop with an RTX 3060 (6 GB) on Windows, a local LiveKit server, and Gemma 4 26B-A4B through Google's API with three keys.
- **Transcripts of DUET's own speech**, which the response-quality and latency scores read, come from faster-whisper here: the benchmark's Parakeet recognizer needs NVIDIA NeMo, which runs on Linux. The tool metrics, including Pass@1, do not depend on this.
- **The LLM judge.** The official judge is gpt-4o. This run was judged by Gemma 4 26B-A4B answering the official prompts unchanged (`bench/judge.py`). Samsung's re-run uses gpt-4o whenever an `OPENAI_API_KEY` is set.

Pass@1 by kind of item

Items	Passed	Pass@1
Difficulty: easy	31 of 36	0.86
Difficulty: medium	30 of 34	0.88
Difficulty: hard	20 of 30	0.67
Disfluency: none	21 of 31	0.68
Disfluency: FILLER	24 of 29	0.83
Disfluency: PAUSE	13 of 18	0.72
Disfluency: SELF_CORRECTION	15 of 17	0.88
Disfluency: FALSE_START	11 of 12	0.92
Disfluency: HESITATION	8 of 10	0.80
Domain: ecommerce_support	23 of 29	0.79
Domain: housing_location	21 of 26	0.81
Domain: finance_billing	23 of 25	0.92
Domain: travel_identity	14 of 20	0.70
State rollback: no	66 of 83	0.80
State rollback: yes	15 of 17	0.88
Tools expected: 1	58 of 66	0.88
Tools expected: 2	13 of 18	0.72
Tools expected: 3	10 of 16	0.62

Where the remaining points are

Every recording that did not pass by exact match, read from its trace and classified:

Kind	Recordings	What happened
Format only	8	The right tool and value in another form: "mechanical keyboard" for "keyboards", "the gym" for "Gym", "Vegas" for "Las Vegas". The benchmark's judge rules accept such differences
Heard wrong	7	The recognizer misheard a spelled-out code or amount ("D, E, L, bye, V" for DELIV; "B0, B1, B2" for BOB12; "\$800" for 1800) or missed a word ("desk" heard as "yes"). DUET acted on what it heard, or asked for what it could not
Provider stalls	3	Every request for one model call went unanswered for 28 s (Google returned 504 Deadline Exceeded), despite backups on every key

Kind	Recordings	What happened
Conditional requests	3	The expected calls contradict the benchmark's own mock data ("book it if a flight is under \$300" expects the booking and the alternative, while the only flight costs \$450); or the condition is the user's own judgement ("if the rate looks good to me")
Not passable as scored	2	The expected city is never spoken; the expected document number differs from the recording
Judgement	2	DUET asked a question instead of acting: once for a gift idea described only as "something in the electronics section", once waiting for a city the user never gave

The first two rows are the ceiling of any agent that listens through a recognizer and is scored by exact match; the judge recovers the format row. The conditional and unpassable rows are properties of the benchmark items. What is left for DUET itself is small: provider stalls and two judgement calls.

Model requests

Every request to Google's API is logged in the run's `agent.log`:

answered	220
backup requests	109
calls answered while backups were pending	38
calls cancelled (plan overtaken)	33
transient errors, retried	221
answer time	median 3.3 s, p90 7.2 s

Transient errors are retried within about 1.5 s, and a request with no answer after 6 s gets a backup on the key with the most room, so a burst of server errors costs seconds, not the recording.

Reproduce this page

```
bash reproduce.sh --force           # a new run: results/live/<time>/
python bench/results_tables.py results/live/<time>  # the tables above, from its reports
```

Engineering notes

How the benchmark scores, what we measured, and the decision each measurement led to.

What we measured while building DUET for Full-Duplex-Bench v3, and the design decision each measurement led to. Every number here comes from a run of the official pipeline or of our offline tools, on the code in this repository.

- 1. How FDB-v3 actually scores
- 2. What the ears hear
- 3. Listening with no model at all
- 4. Turn-taking decisions
- 5. Making a hosted model dependable
- 6. Reproducible on someone else's machine
- 7. A critical review of the benchmark path
- 8. Where today's systems fall short

1. How FDB-v3 actually scores

We read the benchmark's runner and evaluation scripts before writing any agent code.

- **Pass@1 fails on any extra tool call.** Precision must be 1.0: a call made with a stale, pre-correction value fails the item even if the corrected call follows. This is why DUET gates *every* tool call, read-only ones included.
- **Arguments are compared per tool.** Each expected call is matched with the first actual call of the same tool; with `--use-11m` a judge decides whether the values mean the same ("August 20" and "2026-08-20"), otherwise the match is exact after lower-casing.
- **The end of the user's turn** is the end of the last word before the first silence longer than 2 s, as heard by the scoring recognizer. Pauses shorter than that are inside the turn, and agent speech before that point counts as an **interruption**.
- **Every recording is the request followed by about 30 s of the speaker's real room noise** (the shortest tail is 29.7 s). The output recording is exactly as long as the input, so every tool call and every word of the answer must happen inside that window.
- **The recorder adds a near-constant ~1.9 s** to every system's measured latency: agent audio queued during the 2 s before streaming starts is written first. It affects every published baseline the same way.
- **Tool schemas matter.** Some expected calls omit an argument that the reference agents' schema makes required (an apartment search with no budget), or use a filter the reference schema lacks (`pets_allowed`). A model forced to fill a required field invents a value, so DUET keeps the benchmark's names but makes optional what a real API treats as optional.
- **A few items cannot be passed as scored.** In one the expected city is never spoken; in another the expected document number differs from the recording. Three conditional requests expect a branch that the benchmark's own mock data contradicts: *"if there is a flight under \$300, book it; otherwise update my licence"* expects both actions, while the mock's only flight costs \$450. DUET follows the data, as a real agent must; we do not special-case items.

2. What the ears hear

`bench/offline_asr.py` runs the agent's recognizer (faster-whisper large-v3-turbo with Silero VAD) over all 100 inputs.

- **Every expected argument value appears in the transcript for 67 of 100 items.** Most misses are naming, not hearing: "British pounds" for GBP, "driver's license" for `driver_license`, "Vegas" for Las Vegas. The thinker resolves these.
- **About 8 are real mishearings**, mostly in spelled-out codes: a letter "I" heard as "bye", "1800" heard as "\$800", "B-O-B-1-2" heard as "B0, B1, B2". These set the ceiling for any agent that listens through a recognizer.
- **No phantom input from the noise tails.** With VAD gating and the non-speech filter, zero segments were kept from the 30-second ambient tails.

3. Listening with no model at all

`bench/run_live.py --dry-run` runs the whole official pipeline with the agent answering "Okay." to every closed turn. It measures the ears and the turn-taking alone.

Metric (official scripts, all 100 recordings)	Result
Conversations answered (turn-take)	100%
Interruption rate	12.0%
First-response latency, as the benchmark measures it	4.09 s (median 4.07, p90 4.77)

Where the 4.1 s goes (medians):

Stage	Seconds
VAD notices the user stopped	0.71
ASR of the final segment (laptop RTX 3060)	0.62
End-of-turn decision	0.79
TTS to first audio	0.18
Benchmark recorder offset (constant)	1.87

This run is the floor for DUET's own response time, and it is where the turn-taking defects in the next section were found. The interruptions are the end-of-turn model closing a turn inside a pause; with the model, the thinker's `keep_listening` decides such turns instead, so 12% is an upper bound. The complete record is in [results/reported/listening_dry_run](#).

4. Turn-taking decisions

Each rule below exists because a run showed the failure it prevents.

What we saw	Consequence	Decision
LiveKit drafts a reply during a pause; a draft that was then thrown away still marked the request as answered	The real turn found nothing to answer	A request is closed only when a reply that finished thinking is actually spoken (<code>DuetAgent.reply_delivered</code>)
The worker's default load is CPU usage, and the benchmark's scoring recognizer loads the CPU between recordings	The worker refused rooms	Load is counted in conversations
The end of turn fires after about 0.8 s of silence, including after "Like, you know." and "Could you track it for me?" (the order id came next)	A premature, failing call	The thinker decides whether the user has finished (<code>keep_listening</code>); once they have been quiet for 2.5 s it must act or ask
A third of all turns closed before their last words were transcribed	A plan made on the shorter text could act on a stale value	Words that arrive after a turn closed advance the epoch; every call carries its plan's epoch
The same late transcripts started model calls that the late words then voided	Wasted calls against the per-minute budget	Before the first call, the thinker waits up to 1.5 s for a transcript that is still due
A burst of room noise after a request advanced the epoch and voided a correct plan; the noise had no words, so no new turn followed	An unanswered request	Speech makes a call wait, but only new <i>words</i> void a plan; 2.5 s after wordless speech, the plan stands
The first room a worker joins starts WebRTC, which took up to 16 s on one machine, while the runner streams 2 s after joining	The first recording's request was half gone	A throwaway warm-up conversation before the first recording
With a hosted model the thinker needs seconds, and the fixed acknowledgement waited for its decision	Up to 8 s of silence after the user finished	"One moment." once the user has been quiet for 1.6 s. The benchmark ends a turn at 2 s of silence, so this cannot land inside it
The user barges in while a booking is executing	The cancelled reply could leave a performed action unrecorded; a re-plan could repeat it	Past the gate, a call is shielded, runs to completion and is recorded

The effect on the live pipeline: on a three-recording pilot with the model, first response 4.38 s as the benchmark measures it (against the 4.09 s floor above), with no interruptions.

5. Making a hosted model dependable

Gemma 4 26B-A4B through Google's API answers a thinker call in 3.3 s at the median and 7.2 s at the 90th percentile (220 answers in the reported run). Two properties of a hosted model decide whole recordings, because a recording's room closes about 30 s after the request:

- **Per-minute limits.** A project may send 16,000 input tokens per minute per model, and one thinker call is 2,300-2,600 tokens (the instructions and 13 tool definitions), so a single key allows about six calls a minute. Across an earlier full run DUET used about 7,200 input tokens a minute on average, and a busy minute reached about 19,600. *Decisions:* a token budget per key that books every call before it is sent, several keys whose limits add up, and no calls spent on plans that are already void (no preemptive planning; the transcript wait above).
- **Transient errors and stalls.** Some requests fail with a 500 or 503 and come back within about 1.5 s; retries absorb them. A few simply stall with no answer. Waiting for a stalled request, then starting over, could use up a whole deadline. *Decision:* backup requests. A request unanswered after 6 s stays in flight and a second one goes to the key with the most room; the first answer wins. In the reported run, 109 backups were sent, and in 38 calls an answer arrived while backups were pending.

Every request is logged in `agent.log` (key, time, tokens, and every retry, backup and cancellation), so a slow day at the provider can be told apart from a wrong answer.

6. Reproducible on someone else's machine

Only Samsung's re-run of `reproduce.sh` is scored, so the script must work on a machine we have never seen.

- **Lock files for every package**, transitive ones included (`requirements.lock`, `requirements-bench.lock`), resolved for Linux x86_64. `reproduce.sh` uses a pinned `uv` to create Python 3.11 environments, so neither a system Python nor `python3-venv` is needed.
- **One torch for both environments** (2.10, CUDA 12.8 build), which runs on CUDA 12.x and 13.x drivers. Left unpinned, the benchmark's NeMo dependency would pull a CUDA-13-only torch and lose the GPU on a CUDA 12 machine.
- **Pinned models:** the exact Hugging Face revisions of Whisper and Kokoro, verified to load offline from the cache, and the spaCy model Kokoro's phonemizer fetches on first use.
- **Pinned benchmark:** the FDB-v3 commit, the data archive's SHA-256, the LiveKit server version and its checksum.
- **A fresh clone on a clean Linux system** (WSL2 Ubuntu) ran `reproduce.sh` end to end: installation, preflight, the official runner with the Parakeet scoring recognizer, and the three evaluations.

7. A critical review of the benchmark path

We read every file the benchmark exercises and assumed each could be wrong.

Checked and sound:

- **The scorer's rules** match our reading: the exact multiset of tool names, then each expected call against the first actual call of the same tool. No item expects an identical call twice, so the idempotency ledger can never block a required call.
- **The agent joins in time.** Across a full run the agent was listening 1.7 s before streaming began; no request audio was missed.
- **Spoken facts survive the voice.** Typical answers went through Kokoro and back through a recognizer with every id, amount, name and date intact ("XK42Q8", "\$1,850").
- **Late transcripts never strand a plan.** In all 145 cases across two full runs, a new closed turn (121) or new speech (24) followed.
- **The mock backend is stateless**, so one instance shared across conversations caches nothing, and our tool latencies use the same `instant` profile as the reference agents.

Found and fixed:

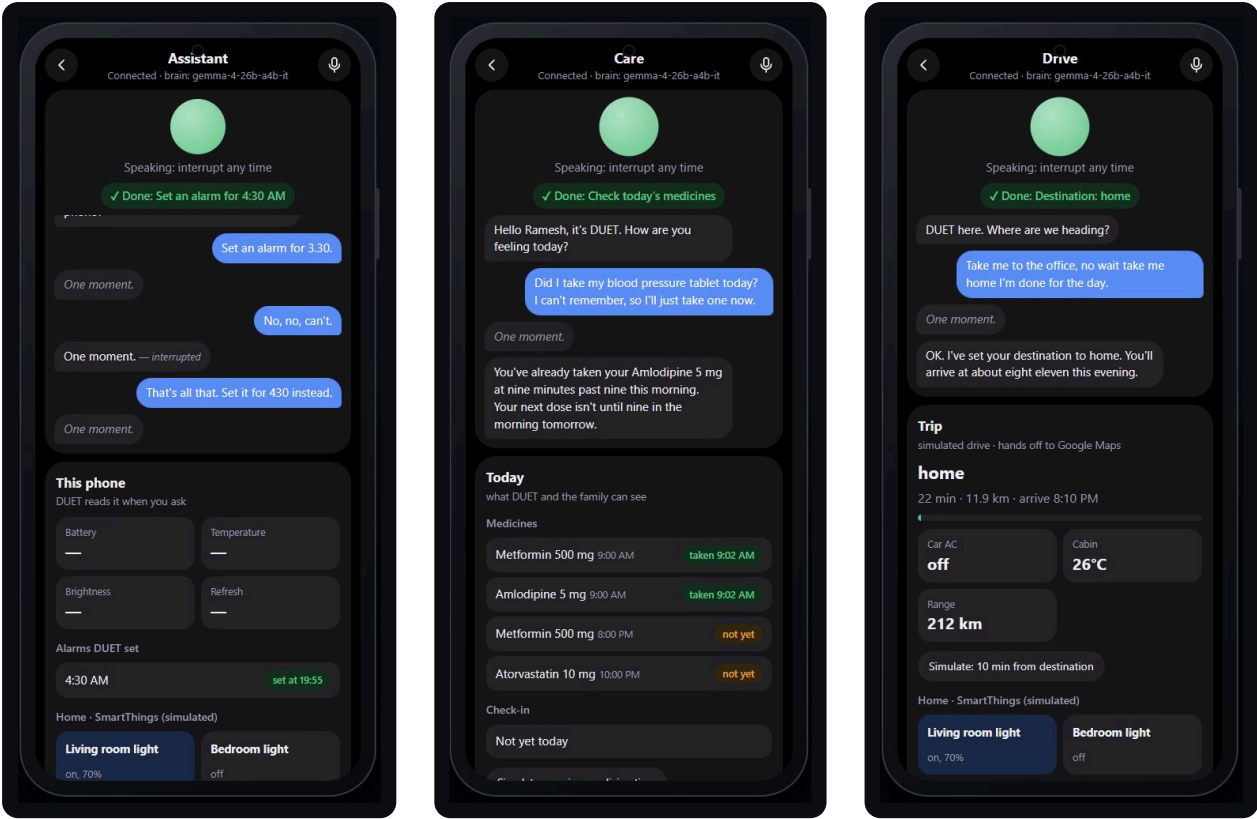
Finding	Risk	Fix
A model can be withdrawn from a key	Every turn an apology	A fallback model, and a preflight that checks every key and a real tool call before the run
A GPU whose libraries fail to load killed the worker at start	No run at all	Whisper and Kokoro fall back to the CPU, same models
Audio was resampled without a low-pass filter	Aliased input to Whisper	LiveKit's band-limited resampler, live and offline
The acknowledgement named values ("checking flights to Paris")	A stale value in the transcript the judge reads, if the user then corrects it	The fast voice never names a value
A tool name the model invents raised an exception	An apology instead of an answer	An error result the model can recover from
An interrupted tool chain kept calling the model	Wasted calls on a void plan	The chain stops at once
The benchmark's fixed <code>/tmp/agent_tool_calls.log</code> may belong to another user on a shared machine	Every tool call becomes an apology	Checked before the run; a logging failure is logged, not fatal
A pre-set data folder, half-finished installs, missing LiveKit credentials	A failed re-run	<code>reproduce.sh</code> reuses existing data, retries downloads, marks finished installs and validates credentials
Markdown or snake_case in an answer	Symbols read aloud	Stripped before speech; underscores spoken as spaces

8. Where today's systems fall short

- **The FDB-v3 paper.** The best published system, GPT-Realtime, passes fewer than 59% of self-correction items. Gemini Live 3.1 gives no spoken reply in 22% of items, although in most of those it had already called its tools, and its pre-emptive calls "lock in the stale destination". The cascaded Whisper → GPT-4o pipeline scores 0.176 on self-corrections, because it finalises the uncorrected transcript.
- **Voice assistants on the road.** User reports of in-car assistants (2026) describe assistants that keep talking after the driver has already chosen on the touchscreen and misreport failures. Sources are in [USE_CASE_RESEARCH.md](#).

Use-case extension: DUET for Galaxy

The same agent, coordinator and model on a Samsung phone.



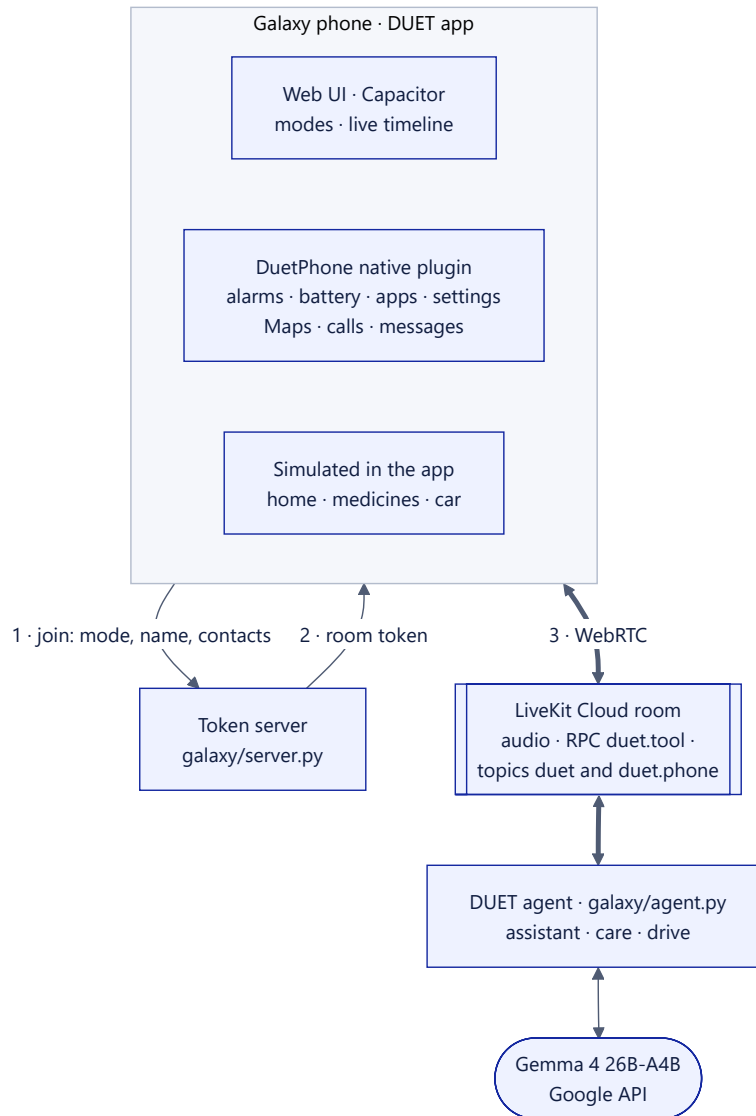
Assistant: "Set an alarm for 3:30... no, 4:30." One alarm, 4:30. Care: DUET checks the record and stops a second dose. Drive: "The office, no wait, home." Only home is set.

Frames from the recorded runs on Gemma 4 (the app's web build, with a scripted user voice; the agent, the model and every tool call are live).

DUET is a full-duplex voice agent: you can interrupt it, correct yourself mid-sentence and change your mind, and it never acts on words you took back. The benchmark submission shows this on Full-Duplex-Bench v3. This app puts the same agent, the same coordinator and the same Gemma 4 model on a Samsung Galaxy phone.

Mode	For whom	What it shows
Assistant	Every Galaxy owner	An alarm corrected mid-sentence; battery troubleshooting from the phone's real readings that you can interrupt; installed apps, brightness and settings, Google Maps, the smart home
Care	An older person living alone, and their family	Medicines with a double-dose guard, calls and messages to family, reminders, a daily check-in, a family alert
Drive	A driver, eyes on the road	A destination changed mid-sentence, arrival-time messages, the car's climate, the home from the car

How it works



- **The agent** is the benchmark's agent with each mode's tools and instructions (`duet_voice/galaxy/modes.py`): the same ears (Silero VAD, Whisper large-v3-turbo, the end-of-turn model), the same mouth (Kokoro), the same fast voice, thinker and coordinator.
- **Every action passes the coordinator:**
 - nothing runs while you are speaking;
 - nothing runs before you have been quiet for the commit hold;
 - a plan your later words overtook is never carried out;
 - no action runs twice.
- **The app shows each step live** on its timeline: what DUET heard, what it planned, **Never ran** (a plan your correction overtook), **Done**, **Not repeated**, and what it said.
- **Context the phone needs:** the thinker knows the time; if you cut an answer short, it knows how much of it you heard; a plan that never ran is reported as never carried out, so DUET never claims it.

What is real and what is simulated:

- **Real, through Android's public interfaces:** alarms and timers in the Clock app; battery, temperature, current drain, brightness, refresh rate, screen timeout, Wi-Fi, Bluetooth and location; screen time per app and the installed apps; brightness and timeout changes; settings screens, opening apps, Google Maps

navigation; calls and texts.

- **Simulated inside the app:** the SmartThings-style home, the medicine schedule and the car's trip.

Run it

1. The laptop or server. Needs `.env.livekit` with your LiveKit Cloud project's URL, key and secret, and a Google API key in `.env.local` (`GOOGLE_API_KEY` or `GOOGLE_API_KEYS`), as for the benchmark.

```
bash scripts/galaxy_demo.sh
```

It checks the Google key, starts the agent (connected to LiveKit Cloud) and the token server with the web app on port 8787, and prints the machine's address for the phone, for example `192.168.1.20:8787`. On Windows, allow Python to accept connections on private networks the first time, or the phone cannot reach it.

2. The phone.

1. Install the APK (`app/android/app/build/outputs/apk/debug/app-debug.apk`): copy it over and open it, allowing "install unknown apps" once, or use `adb install -r app-debug.apk`.
2. Put the phone on the same Wi-Fi, or pair in one step: open `localhost:8787/pair.html` on the laptop and scan its code from the app.
3. Open DUET, then Settings:
 - enter the server address, your name and the contacts DUET may call;
 - under Phone access, allow **Screen time per app** and **Change brightness and timeout**.
4. Choose a mode and allow the microphone. Galaxy Buds give the cleanest audio: no echo and no false interruptions.

In a browser instead: open `http://localhost:8787` on the laptop. Phone actions are then simulated.

Try these

Mode	Say	What to watch
Assistant	"Set an alarm for 3:30." Pause. "No no, cancel that, set it for 4:30."	Timeline: 3:30 Never ran , 4:30 Done ; one alarm in Clock
Assistant	"Set an alarm for 3:30", wait for DUET, then "No, make it 4:30"	DUET had set 3:30, so it cancels it and sets 4:30
Assistant	"Wake me at 6:30, or maybe 7, hmm."	DUET asks "6:30 or 7?" instead of guessing
Assistant	"My battery drains super fast. What's going on?" Then, while it explains: "Oh wait, I think it's my brightness and games."	Real readings; DUET stops, checks your theory against the numbers, and offers a fix without making it
Assistant	"How many shopping apps do I have?"	Counted from the phone's real app list
Care	"Did I take my blood pressure tablet? I'll just take it now."	The double-dose guard: it was taken this morning, so not again
Care	Tap "Simulate: evening medicine time"	DUET speaks first with the reminder
Drive	"Take me to the office. No wait, the airport."	Only the airport is set

Mode	Say	What to watch
Drive	Tap "Simulate: 10 min from destination"	DUET offers one useful thing, such as cooling the house

Test without a phone

`scripts/galaxy_fake_phone.py` joins a room like the app does. It speaks scripted lines, answers DUET's tool calls with a simulated phone, and saves every timeline event to `results/galaxy/scenario_*.json`:

```
python scripts/galaxy_fake_phone.py --list
python scripts/galaxy_fake_phone.py alarm_late
```

Build the APK

JDK 21, Android SDK 36 and Node 20+:

```
cd app
npm install
npm run sync
cd android
./gradlew assembleDebug
```

Record the demo video

The film `results/galaxy/video/DUET_for_Galaxy_demo.mp4` is made from live runs of the real stack: Gemma 4, the agent and LiveKit Cloud. Only the user is scripted:

- the lines in `app/www/demo/scenes.json` are spoken by Kokoro voices (`app/www/demo/audio/`);
- `demo-driver.js` plays each line into the room instead of a microphone. It waits for DUET's reply, or cuts in while DUET is talking, as a person would;
- `demo.html` is the 1920×1080 stage: the app in a phone frame, DUET's steps live, and captions.

```
python scripts/galaxy_demo_voices.py           # once: the scripted user's lines
bash scripts/galaxy_demo.sh                     # the agent and token server (see above)
cd app && npm install
TAKE=1 node tools/record_demo.js all             # one take of every scene: <scene>_t1.mp4
cp ../results/galaxy/video/correction_t1.mp4 ../results/galaxy/video/correction.mp4 # the take to
keep, per scene
node tools/build_film.js                        # cards + scenes -> DUET_for_Galaxy_demo.mp4
```

Each take is a new live conversation, so takes differ: record two or three and keep the best of each scene. Every take's full log is in `results/galaxy/traces/<room>.jsonl`; the room name is printed when the take is recorded.

Limitations and next steps

What we still lose, and where DUET goes next.

Limitations we measured

Latency	First response 4.8 s; key information spoken at 14.4 s (GPT-Realtime: 6.9 s). A hosted model call takes 3.3 s at the median and 7.2 s at the 90th percentile.
Provider stalls	3 of 100 recordings were lost to model calls Google never answered (504 Deadline Exceeded), despite backups on every key.
Hearing	Spelled-out codes are misheard ("D, E, L, bye, V" for DELIV): 7 of 100 items.
Response quality	0.73 against GPT-Realtime's 0.79: the answers are correct but plain.
Our scoring	Judged by Gemma 4 standing in for gpt-4o; 75 of 100 pass by exact match with no judge at all. Samsung's re-run uses gpt-4o and Parakeet.
The extension's demo	Recorded on the app's web build; the home, the medicine schedule and the car are simulated.

Next, as a PRISM worklet

1. **On-device Gemma on Galaxy.** No round trip and no key limits: the answer's latency drops, and the user's words never leave the phone.
2. **Confirm before acting on codes.** Recognizer confidence per word; a low-confidence order id is read back before the tool runs.
3. **Speculative reads, gated writes.** Start safe lookups while the user is still talking; keep every state change behind the commit gate.
4. **SmartThings and Android Auto.** The Care and Drive modes on real devices, with Knox-consented health reads.
5. **Field metrics.** Stale-action rate, duplicate-action rate and time to first feedback, measured on real users.

Appendix: use-case research

The research behind DUET for Galaxy.

Before building the extension we asked one question: not "what is a good demo", but **where is a real-time, full-duplex agent that acts the reason a Samsung product gets better**, as opposed to one more voice front end on an ordinary assistant. This document records the research, the candidates we compared, and why we built what we built (section 6).

1. The test a use case has to pass

DUET's contribution is narrow and specific. It is not speech recognition, a voice, or a bigger model; those are commodity. It is *acting correctly while a conversation is still moving*:

- **Commit timing.** It never acts on a half-said or corrected value (commit gate, epochs).
- **Exactly-once actions.** It never performs a state-changing action twice (idempotency ledger).
- **Never silent, never lying.** A fast mind speaks while a slow mind works, and nothing is announced as done before the tool says so.
- **Keeps talking while work runs.** Tools and background tasks proceed while the conversation continues, and the agent recovers when they are slow or fail.
- **Interruptible at any moment.** The user can cut in, redirect, or take over.

So a use case earns DUET only if **most** of these hold:

1. **Voice is the primary or only safe channel.** The user's hands or eyes are busy. If a screen is just as easy, full-duplex is a nicety.
2. **Actions are consequential and multi-step.** They send, book, pay, unlock, or change devices, not just answer questions.
3. **People revise mid-sentence.** The domain naturally produces corrections, second thoughts and parallel requests.
4. **Tools are slow or unreliable.** Network, devices or backends lag or fail, so the conversation must continue around them.
5. **The assistant must keep talking while acting, and yield instantly.**

We score every candidate against these five.

2. What exists today (facts, with sources)

Fact	Why it matters	Source
Bixby 4.0 (One UI 8.5, launched 31 Mar 2026, rolling out on the Galaxy S26) was rebuilt around an LLM. Device functions became "callable agents", it plans multi-step tasks, controls SmartThings ("start cleaning the floor"), and uses Perplexity for web answers. It has a "Bixby Live" interface.	Samsung already ships <i>agentic</i> device control. DUET cannot win on "an LLM that calls device functions". It has to win on doing that under real speech : corrections, interruptions, background work, truthfulness.	TechBuzz, Droid Life, The Meridiem

Fact	Why it matters	Source
Nothing public shows Bixby 4.0 handling mid-sentence corrections, interruptions during actions, or continued conversation while tasks run.	This is the gap DUET targets. It is unverified, so we should demonstrate it, not assert it.	same
Gemini on Android Auto (2026) reaches 250M+ vehicles. Users report it "won't stop talking", keeps talking after the driver has already chosen on the touchscreen, is inaccurate, and shows a "willingness to lie when it fails".	These complaints describe DUET's design points: brevity from the talker, yielding when the user takes over, and never claiming success without a result.	Android Authority, 9to5Google, Google blog
The FDB-v3 paper : Gemini Live 3.1 scores 0.353 Pass@1 on self-corrections, stays silent in 22% of cases (tools ran, nothing was said), and its pre-emptive calls "lock in the stale destination". GPT-Realtime leads at 0.600.	Independent evidence that today's best realtime models fail exactly where DUET intervenes. Our own FDB-v3 numbers will be the proof.	arXiv 2604.04847
The Gemini Live API has non-blocking (asynchronous) function calling with response scheduling (interrupt, when idle, or silent).	"Keep talking while a tool runs" is becoming a platform primitive. DUET's value is <i>when</i> to commit and exactly-once execution, not just concurrency.	Google Cloud docs
Android 16 AppFunctions : apps expose functions to agents (Samsung Gallery does on the S26). <i>Calling</i> them needs <code>EXECUTE_APP_FUNCTIONS</code> , which is privileged or role-restricted, with Gemini in private preview.	A store-installed DUET app cannot call Samsung apps' agent functions today. A Samsung-internal DUET could.	Android docs, Android Developers Blog
Any third-party app can be set as the default digital assistant and be summoned with the Galaxy side button. ChatGPT does this.	DUET can live exactly where Bixby and Gemini live on a Galaxy phone, with no special access.	SamMobile, Samsung support
SmartThings public REST API (personal access tokens, OAuth) with virtual devices (<code>smartthings virtualdevices:create</code>). Several community MCP servers already expose it to LLMs.	DUET can control the same Samsung home device graph that Bixby controls, through a supported API, and the changes appear in Samsung's own SmartThings app.	SmartThings docs, SmartThings CLI, community
Samsung x Hyundai/Kia : the 2026 Android Automotive infotainment integrates SmartThings. "Car-to-Home" controls home appliances from the car; "Home-to-Car" checks and conditions the car from the phone.	Samsung is actively building the car→home bridge. A conversational layer for it is directly on Samsung's roadmap.	Samsung Newsroom, Hyundai, SamMobile
HARMAN (a Samsung subsidiary) ships the Ignite in-vehicle platform, on which OEMs deploy voice agents (Bixby, Alexa and others).	A realistic path from prototype to product for an in-car orchestration layer.	Tech Monitor, SoundHound x HARMAN
Samsung SDS AI Contact Center runs voicebots that answer inquiries, screen applications and process requests. In the wider market, voice AI handles 19% of inbound contact-center volume (2026), and barge-in is a named pain point.	A B2B line where "don't double-book, don't talk over the caller, hand off cleanly" is money.	Samsung SDS AICC, Maven AGI, Decagon
Galaxy XCover7 Pro / Tab Active5 Pro target frontline workers, with push-to-talk, glove and wet-touch support.	Hands-busy enterprise users with procedures, readings and orders.	Samsung Newsroom

Fact	Why it matters	Source
Gemma 4 (April 2026): E2B, E4B, 26B MoE and 31B dense, with native function calling.	A fully local "offline DUET" fits in 48 GB, in line with Samsung's on-device AI direction.	Google

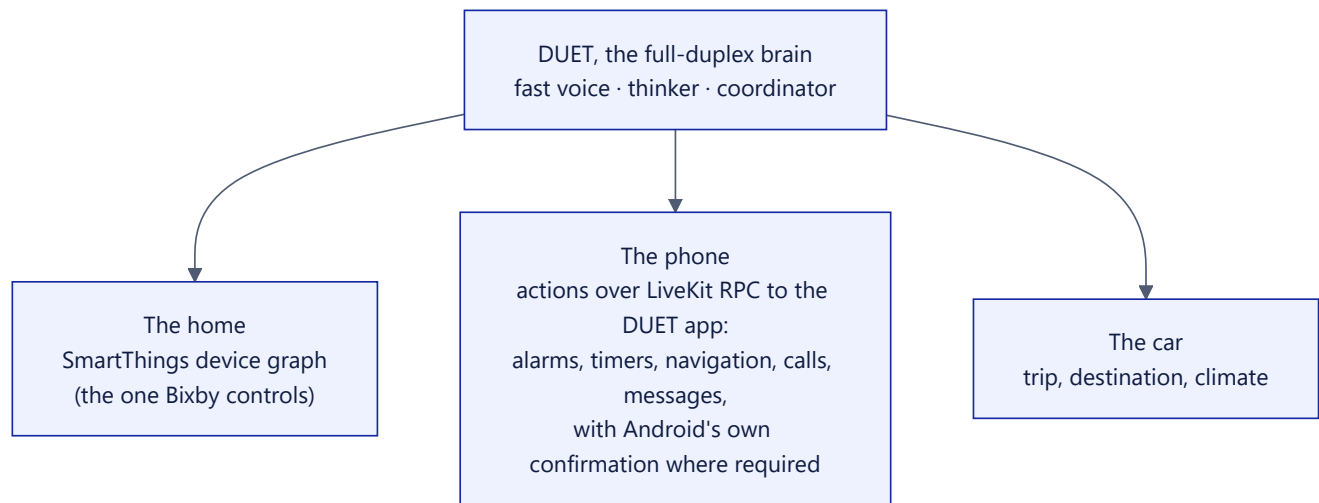
3. Can DUET work alongside Bixby?

Direct integration: not with public means today.

- There is no public API for a third-party app to send commands to Bixby 4.0 or to call its device agents.
- The AppFunctions caller permission is privileged.
- Bixby Capsules (2019 Developer Studio) run the other way: Bixby calls a developer's service, not the reverse.
- Samsung Modes & Routines has an undocumented content provider that can run a routine by UUID. Tasker users found it; it is not a supported API.
- Driving Bixby's UI through an accessibility service is fragile and against Play policy.

None of these is a sound foundation for a product.

What is feasible, and close to the idea. Split the system the same way the idea does, using supported mechanisms:



- **On the phone.** DUET is the Galaxy's default digital assistant (side button). The phone is the ears and mouth, and executes device actions on request: the agent calls the phone over LiveKit RPC, which LiveKit supports for exactly "forward LLM function calls to your client". This is the "Bixby executes device-level actions" role, played by the DUET app itself.
- **At home.** SmartThings devices, virtual or real, are driven by DUET, and every change shows up live in Samsung's own SmartThings app. Bixby 4.0 would control the same graph.
- **Inside Samsung (the product story).** DUET is not a competing assistant but a **layer**. Its coordinator (commit gate, ledger, failure policy) and fast-voice/thinker split can wrap Bixby 4.0's callable agents unchanged; the binding to Bixby needs Samsung-internal access.

Prototype vs product. The prototype runs today with public mechanisms (Android interfaces over LiveKit RPC). A product version would replace the phone-RPC layer with Bixby's callable agents or AppFunctions, which Samsung controls. Nothing in the demo pretends to be Bixby.

4. Candidate directions

The score is how many of the five conditions from section 1 hold (✓ strong, ~ partial, ✗ weak).

A. DUET Co-Driver: in-car assistant with Car-to-Home orchestration

- **User problem.** A driver can only use voice. Plans change mid-drive: stops, destinations, who to tell, what to do at home. Tasks span the car (navigation, charging), the phone (messages, calls, calendar) and the home (AC, lights, garage, vacuum).
- **Existing solutions fall short.**
 - Gemini on Android Auto is conversational but, per 2026 user reports, chatty, doesn't yield, and misreports failures.
 - Bixby and Gemini don't do Car-to-Home orchestration by conversation.
 - Hyundai/Kia's Car-to-Home is touch and voice-command, not a conversation.
- **Why full-duplex.** The agent talks (guidance, confirmations) while listening. Drivers interrupt constantly. Passengers speak and are not addressing the assistant. Actions run while driving continues.
- **Why DUET.**
 - It never acts on a half-said destination ("the Starbucks on 5th, no, the one by the office").
 - It never sends the ETA message twice.
 - It keeps speech to one short line (the exact complaint about Gemini), and stops the moment the driver speaks or takes over.
 - It holds a background task, "when I'm ten minutes out, turn on the AC and open the garage", across the rest of the conversation.
 - It recovers from dead zones (tool timeouts) without lying.
- **Tools.**
 - Routing and places (open map services), weather, SmartThings (real API with virtual devices).
 - Simulated: messages and calendar.
- **Samsung relevance.** Very high: HARMAN Ignite, Hyundai/Kia × SmartThings, Galaxy as the in-car device.
- **Business value.** Driver-distraction reduction is a regulatory and brand-level issue for OEMs, and conversational orchestration is a sellable platform feature for HARMAN.
- **Demo potential.** Excellent: a map that re-routes live, the home devices changing in the real SmartThings app on a phone in frame, the driver cutting the agent off mid-sentence.
- **Complexity.** Medium-high: a dashboard UI, several APIs, and background triggers on a simulated drive.
- **Weaknesses.**
 - It is not in a real car; it runs on a simulated route (fine if said).
 - The in-car space is crowded (Google, Cerence, SoundHound, Mercedes).
 - Safety claims must stay modest.
- **Score.** 1 ✓ · 2 ✓ · 3 ✓ · 4 ✓ · 5 ✓.

B. Galaxy DUET: the full-duplex layer next to Bixby on the phone

- **User problem.** Everyday multi-step phone tasks said naturally while doing something else: "Set a timer for twelve, no, fifteen minutes. Text Priya I'll be late. Turn on the hall lights. And what's the weather at six?"
- **Existing solutions fall short.** Bixby 4.0 and Gemini Live now act on devices, but there is no evidence they handle corrections mid-action or keep talking while tasks run. FDB-v3 shows even the best realtime models lose 40–65% of self-corrections.
- **Why full-duplex.** Moderate. A screen is available, so voice is a convenience unless the user is busy.
- **Why DUET.** Exactly-once actions and commit timing matter because the actions are real (messages, alarms, devices).

- **Tools.** Android intents via LiveKit RPC (a small native plugin in a Capacitor app) and SmartThings.
- **Samsung relevance.** Very high, and the closest to "DUET alongside Bixby".
- **Demo.** Good on a Galaxy S26: side button, and the phone actually sets the timer.
- **Complexity.** High in our window: native Android intents through RPC, permission and confirmation flows, and device testing.
- **Weaknesses.**
 - Android requires user confirmation for SMS or calls from third parties.
 - It can look like "yet another assistant" competing with Bixby.
 - The deepest integration (Bixby agents) is not publicly possible.
- **Score.** 1 ~ · 2 ✓ · 3 ✓ · 4 ~ · 5 ~.

C. SmartThings home (the original recommendation)

- **User problem.** Multi-device routines, spoken naturally, with appliances that take time (washer, oven) and devices that fail (offline).
- **Existing solutions.** Bixby 4.0 and Alexa+ already do LLM smart-home control. DUET's edge is the full-duplex behaviour: background appliance cycles while conversing, recovery when a device is offline, and a human-in-the-loop confirmation for the door lock.
- **Feasibility.** High (SmartThings API with virtual devices).
- **Samsung relevance.** High.
- **Demo.** Good (the real SmartThings app in frame), but less dramatic than a car. At home a screen is usually nearby, so the case for "voice must work" is weaker.
- **Score.** 1 ~ · 2 ✓ · 3 ~ · 4 ✓ · 5 ~.

D. Samsung service line: appliance support and technician booking (Samsung SDS AICC)

- **User problem.** A customer calls about a washer error. They spell a model number, correct themselves ("it's the dryer, not the washer"), need troubleshooting, then a technician slot, maybe parts. Often they are standing at the appliance with their hands full.
- **Existing solutions fall short.** IVRs and today's voicebots talk over callers, re-ask, double-book, and fail at handoff.
- **Why DUET.** Commit timing on bookings, exactly-once scheduling, truthful progress while backends are slow, and a clean human handoff.
- **Business value.** The clearest of all (containment and handle time), and Samsung SDS sells AICC.
- **Weaknesses.**
 - It sits closest to the benchmark's own style (support and booking), so it reads as "more of the benchmark".
 - It is a phone call, not a Samsung device experience, so there is less "wow".
- **Score.** 1 ~ · 2 ✓ · 3 ✓ · 4 ✓ · 5 ✓.

E. Kitchen / Bespoke AI appliances (Samsung Food + oven + Family Hub)

- **Fair challenge.** Timers and recipe step-through already work well with single commands on today's assistants. Full-duplex adds interrupting a read-out ("wait, how much butter?") and parallel timers. That is useful but marginal.
- **Where it gets strong.** Orchestrating appliances ("preheat to 200, no, 180 fan") plus a shopping list plus substitutions via web search. Even then, a screen (Family Hub) sits right there.
- **Verdict.** A pleasant demo, weak necessity. Not our lead.
- **Score.** 1 ✓ · 2 ~ · 3 ~ · 4 ~ · 5 ~.

F. Frontline worker on Galaxy XCover / Tab Active

- **User problem.** A technician with gloved hands follows a procedure, reads values aloud, logs them, orders a part, and asks questions mid-step in a noisy environment.
- **Why DUET.** Commit timing on logged readings ("42.5, no, 45.2 psi"), exactly-once work orders, and long lookups that shouldn't block the procedure.
- **Assessment.**
 - Business value: real (enterprise, Knox).
 - Demo: plausible but abstract for judges.
 - Tools: all mocked (no public work-order system).
- **Score.** 1 ✓ · 2 ✓ · 3 ✓ · 4 ~ · 5 ✓.

G. Accessibility-first phone control (blind, low-vision or motor-impaired users)

- **Assessment.**
 - Social value: the highest. Interrupting long read-outs is central to screen-reader use, and corrections are frequent.
 - Feasibility: poor in our window. Deep phone control needs accessibility services (a policy and ethics minefield), and a credible demo needs real users.
- **Verdict.** A strong "what we'd do next" story, not the demo.

H. Wearables and TV (Galaxy Watch, Buds, Vision AI TVs)

- **Assessment.** These are channels rather than use cases. Buds are the natural audio front end for B or A; a TV has weak necessity. Mention them as extensions of the chosen direction.

5. Summary

Direction	Five-condition fit	Samsung relevance	Feasible in the hackathon	Demo impact	Main risk
A. Co-Driver with Car-to-Home	5/5	Very high (HARMAN, Hyundai/Kia × SmartThings)	Yes, simulated drive	Very high	Scope; crowded market
B. Galaxy DUET next to Bixby	3/5	Very high	Partly (Android native work)	High on a real S26	Time; Android confirmation flows
C. SmartThings home	3/5	High	Yes	Medium	"Bixby 4.0 already does this"
D. Samsung service agent (AICC)	4/5	High (SDS)	Yes	Medium	Too close to the benchmark
E. Kitchen	2/5	Medium	Yes	Medium	Weak necessity
F. Frontline	4/5	Medium-high	Yes (mocked)	Medium-low	Abstract for judges
G. Accessibility	4/5	High	No	High	Feasibility and ethics

The shape of a strong answer. A plus the useful half of B and C:

- A single DUET brain.
- The phone as its ears and mouth (Galaxy default assistant).
- The **car as the stage**, because that is where voice is mandatory and today's assistants are measurably failing.
- **Samsung's own SmartThings graph as the reach into the home.**

It shows every DUET property under real pressure, sits on a live Samsung initiative (Car-to-Home), and keeps the Bixby story honest: DUET is the full-duplex layer Bixby's agents could run under.

6. What we built

DUET for Galaxy: one DUET brain, with the Galaxy phone as its ears, mouth and hands, in three modes that each put a different pressure on the design.

Mode	From the analysis	What it proves
Assistant	Direction B, the full-duplex layer next to Bixby	Corrections mid-sentence on real phone actions (alarms, brightness, apps, Maps, calls, messages); troubleshooting from the phone's real readings that the user can interrupt
Care	Condition 1 at its strongest: an older person for whom voice is the natural channel	Consequential, exactly-once actions: a double-dose guard on medicines, family calls and alerts, reminders, a daily check-in
Drive	Direction A, the co-driver with Car-to-Home	Eyes-busy use: a destination changed mid-sentence, arrival-time messages, the car's climate and the home from the car

Real Android interfaces carry the phone actions (alarms, battery and screen readings, installed apps, brightness and timeout, settings, Maps navigation, calls, messages). The home, the medicine schedule and the car are simulated inside the app, and the demo says so. The same coordinator gates every action, and the app's live timeline shows each step as it happens. Build, run and demo instructions: [app/README.md](#).